UNIVERSITY OF
PLYMOUTH

In collaboration with Peninsula College

School of Technology & Engineering

BSc (Hons) Computer Science (Cyber Security)

(N/0613/6/0002)(04/2027)(MQA/PA15604)

MAL2018 – Information Management & Retrieval (CW2)

Author:

BSCS2509254

Module Leader:

Dr. Ang Jin Sheng

DEC 14, 2025

Table of Contents

Word Count : 1985 words (excluding Captions, References and Appendix)

CODE REPOSITORY

Github	: https://github.com/OoiWeiChyeh/MAL2018_AssesmentModule.git

1.0 Introduction

Coursework 2 (CW2) is built on the database design completed in Coursework 1 (CW1) where established the normalized schema for the TrailService microservice. Previously, raw data processed through systematic normalization (UNF to 3NF) and produced three Entity Relationship Diagrams that created foundation for this phase. Thus, coursework progress to functional RESTful API that shows the CW1 database through standardized HTTP (Hypertext Transfer Protocol) endpoints.

Moreover, system architecture traces what Richardson et al. (2018) describe as the microservices pattern like TrailService operates independently with its datastore. This may shows coordination overhead but may provide clearer service boundaries and reduced coupling compared to monolithic alternatives. The API itself adheres to RESTful principles by using standard HTTP methods (GET, POST, PATCH, DELETE) and JSON for data interchange.

2.0 Background

The Trail Application aims to be a wellbeing-focused platform encouraging outdoor activity. Somehow, though whether such platforms actually promote sustained behavioral change remains debatable. The system splits into few microservices rather than maintaining one codebase. This fragmentation might seem more moving parts typically means more complexity but Newman (2021) suggests it offers practical advantages in terms of independent service evolution and deployment flexibility.

Following what Baeldung (2024) calls the database-per-service pattern, each microservice maintains its datastore. TrailService specifically handles trail metadata like descriptions, coordinates, difficulty ratings etc. Cross-service queries become more complicated and maintaining data consistency across services requires additional coordination.

Relevant Software (2025) reports that the cloud microservices market grew from £1.16 billion in 2023 and projects it reaching approximately £4.56 billion by 2030. Whether this growth reflects genuine technical superiority or simply follows industry momentum is worth questioning but adoption rates suggest organizations find value in the approach.

The implementation uses RESTful API design patterns aligned with guidelines from Microsoft's Azure Architecture Center (EdPrice-MSFT, 2023). Endpoints follow standard conventions: GET for retrieval, POST for creation, PATCH for partial updates, DELETE for removal. JSON serves as the data interchange format throughout essentially a *de facto* standard in modern web APIs, though alternatives like Protocol Buffers or MessagePack might offer performance advantages in certain scenarios.

3.0 System Design and Architecture

The architecture can be understood through three distinct layers. At the foundation sits the database layer, CW1 schema implemented in Azure SQL Edge running in Docker for development purposes. This includes the USER, TRAIL, TRAIL_FEATURE, and TRAIL_LOG tables established during the normalization process.

Above that, ORM (Object-Relational Mapping) layer uses SQLAlchemy to bridge between Python objects and database tables. The models.py file defines classes mirroring the database schema such as User, Trail, and TrailFeature. Each class includes column definitions matching the field definition grids from CW1, plus relationship definitions that SQLAlchemy uses to handle foreign keys and cascade operations.

However, API layer built with Connexion framework sits at the top. Connexion is essentially Flask with OpenAPI specification integration that automatically validating requests against the swagger.yml schema before they reach controller code. The API contract could becomes explicit and versioned though it requires maintaining synchronization between schema and implementation.

3.1 UML Diagrams

Logical Entity Relationship Diagram (ERD)

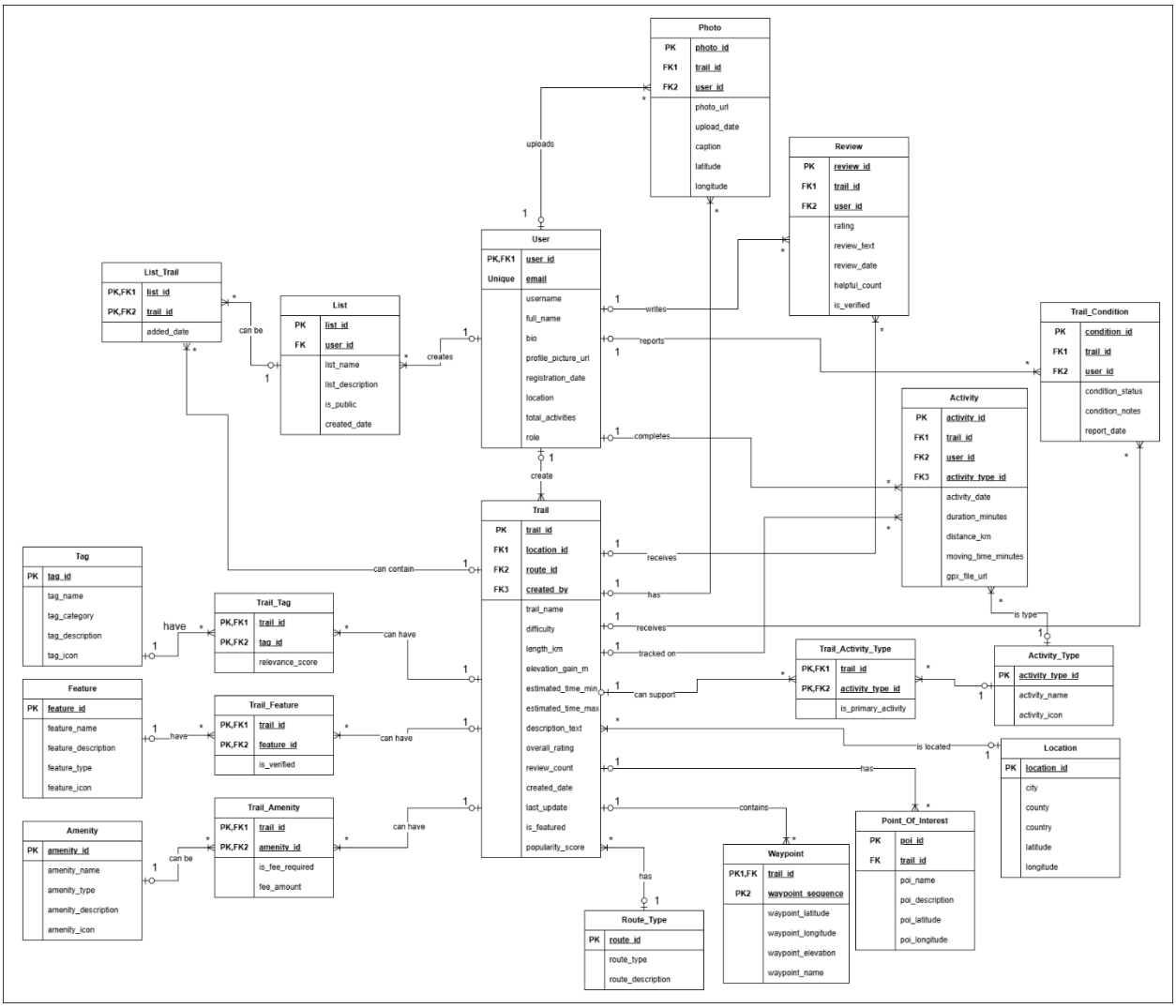

Figure 1 : ERD implemented from CW1

Class Diagram

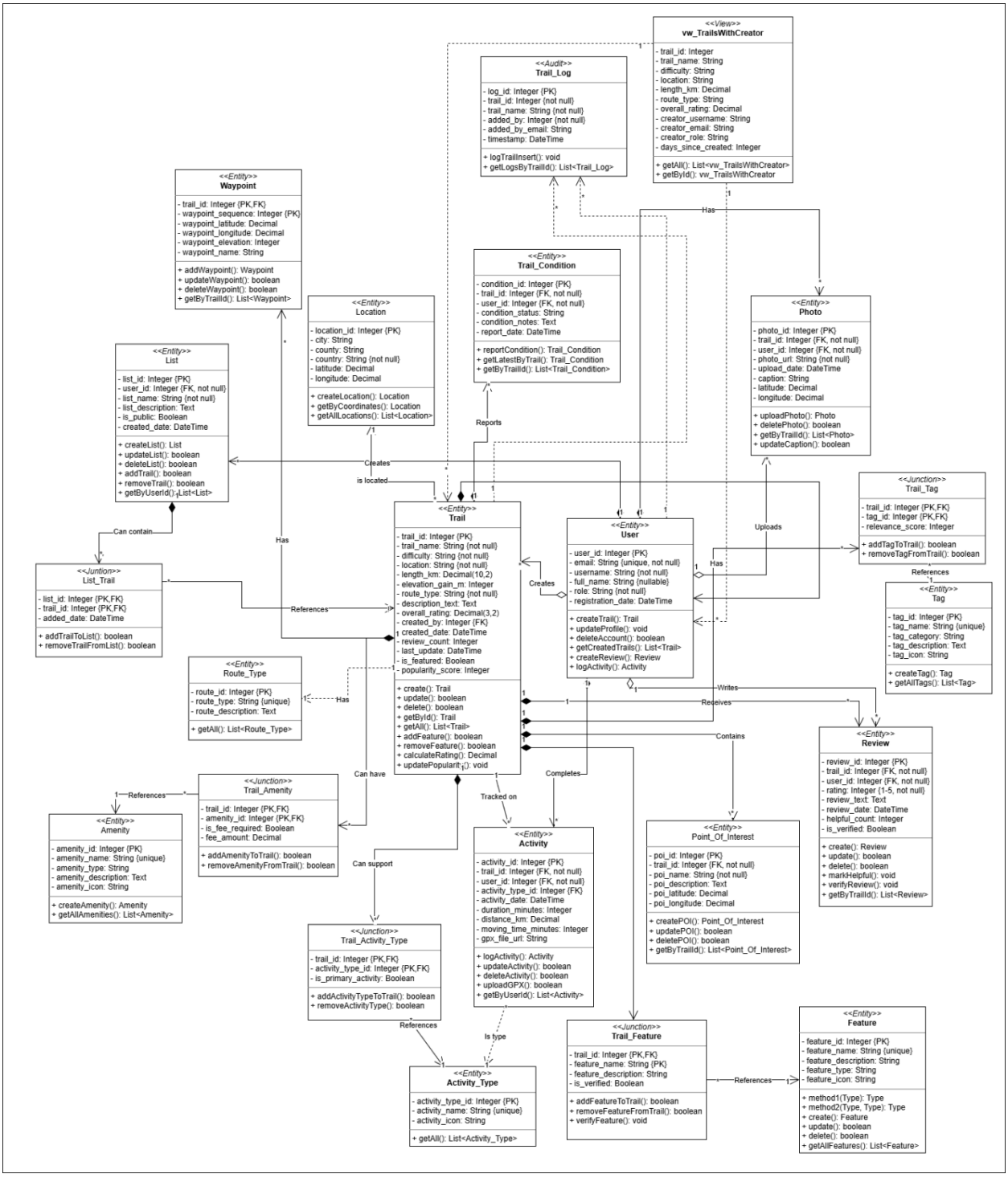

Figure 2 : Class Diagram CW2

Use Case Diagram

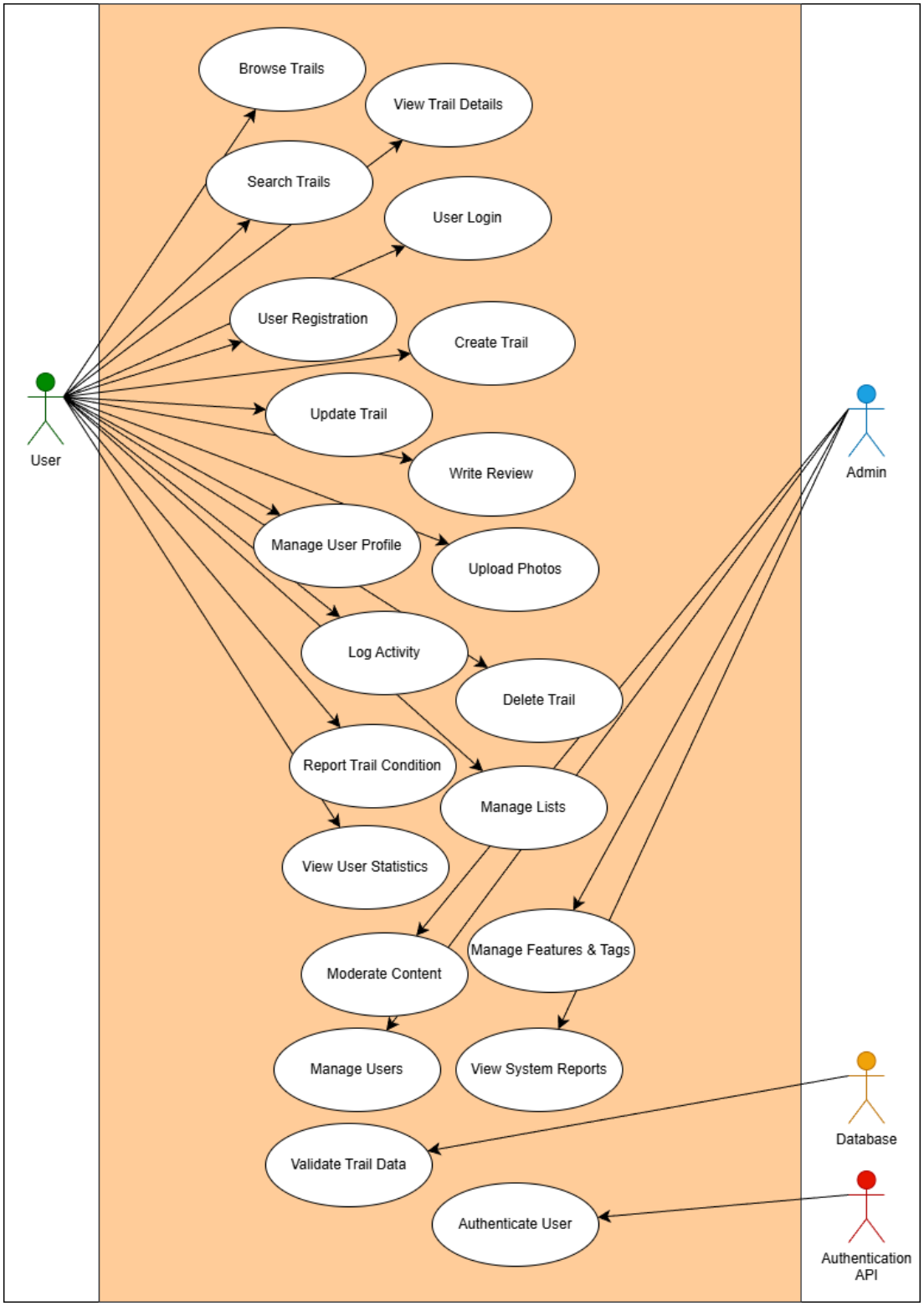

Figure 3 : Use Case Scenario CW2

4.0 Legal, Social, Ethical and Professional Issues

Information Privacy

The system handles what could be considered personal information like user emails, usernames and full names. Under Malaysia's Personal Data Protection Act 2010 (PDPA), this data requires appropriate protection measures and clear consent mechanisms. The current implementation has limitations here. User emails are required for trail creation, linking trails to creators but there's no mechanism for users to request data deletion or modification.

Furthermore, Malaysia's PDPA mirrors several GDPR principles including right to access and correct personal data. If a user requests deletion, their trails presumably should remain but perhaps be anonymized or transferred to a generic "deleted user" account. The foreign key constraint from TRAIL to USER (created_by) currently prevents deleting users who have created trails which provides referential integrity but complicates compliance with deletion requests.

Data Integrity

Foreign key constraints enforce referential relationships. For example, trails cannot reference non-existent users and the database prevents orphaned records through CASCADE DELETE behavior. CHECK constraints validate enumerated values like difficulty and route_type at the database level prevent invalid data even if application validation fails.

The stored procedures from CW1 Exercise 6 provide additional integrity layer. sp_InsertTrail, sp_UpdateTrail, and sp_DeleteTrail encapsulate business rules at the database level though the current API implementation bypasses these procedures in favor of ORM operations. ORM code is more maintainable and testable but stored procedures offer performance advantages and enforce rules regardless of which application accesses the database.

Nonetheless, transaction handling through SQLAlchemy's session management checks atomic operations. When creating a trail, the user lookup, trail insertion and feature association either all succeed or all fail together. The session.commit() call makes these changes permanent while any exception triggers an automatic rollback. This ACID (Atomicity, Consistency, Isolation, and Durability) guarantee is fundamental to data integrity though distributed transactions across

multiple microservices introduce complications that aren't present in this single-database implementation (Kleppmann, 2017).

Security Measures

Any client can create, modify or delete trails. In a realistic deployment, trails should only be modifiable by their creators or administrators which requires integrating with an authentication service and implementing role-based access control.

SQL injection prevention is handled through parameterized queries. SQLAlchemy's ORM generates parameterized SQL automatically and the raw SQL in views.py uses named parameter binding (:trail_id) rather than string concatenation. These has been standard practice since the OWASP guidelines first highlighted SQL injection as a critical vulnerability (OWASP, 2017).

The database connection in config.py includes TrustServerCertificate=yes which bypasses SSL certificate validation. This is acceptable for local development with self-signed certificates but would be a serious security issue in production. Real deployments require proper SSL/TLS certificates and certificate validation to prevent man-in-the-middle attacks.

However, API rate limiting is entirely absent. A malicious client could overwhelm the service with requests that may causing denial of service for legitimate users. Production APIs typically implement rate limiting at multiple levels like per IP address, per authenticated user and globally (Alspach, 2019). The Connexion framework doesn't provide this functionality maybe need additional middleware or a reverse proxy like Nginx.

5.0 Implementation and Testing

The following test cases (A1–A18) were executed to validate that the TrailService REST API fully complies with the OpenAPI 3.0 specification defined in swagger.yml. Together, these tests show CRUD behaviour, schema validation, relationship handling, database view integration and error management.

TC-A1: Retrieve All Trails

Endpoint: `GET /api/trails`

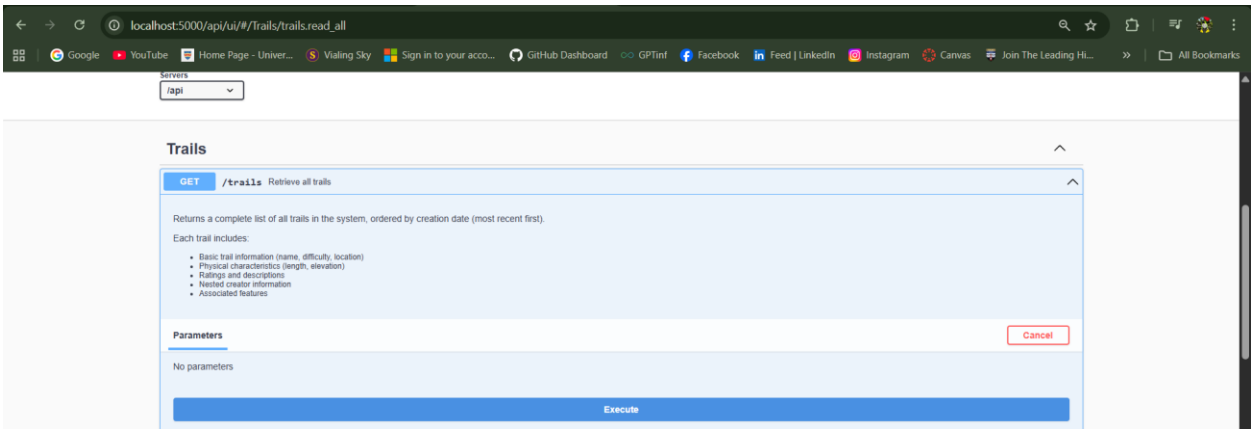

Figure 4 : TC-A1(a)

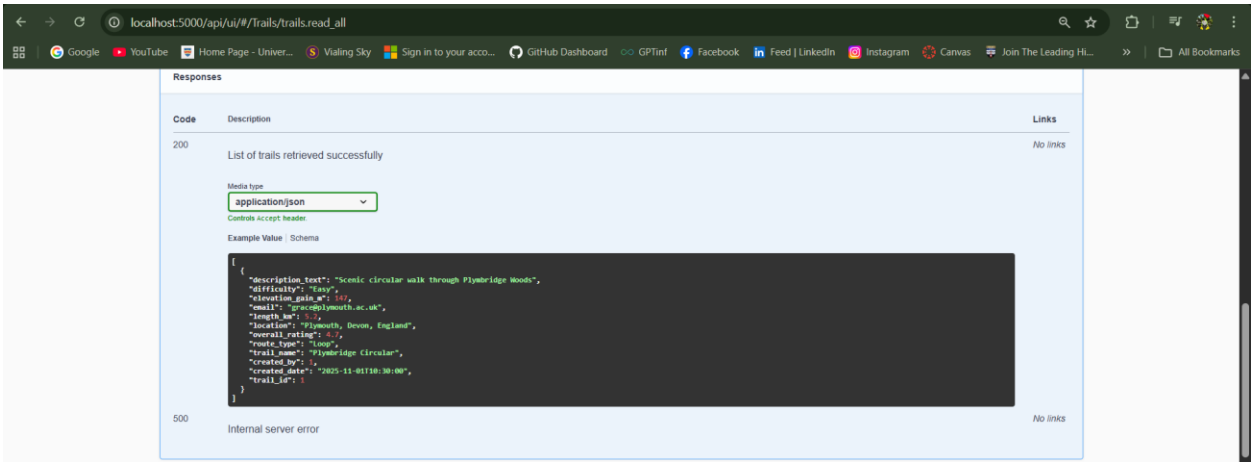

Figure 5 : TC-A1(b)

Purpose

To verify that all trails stored in the system are retrieved successfully and ordered by creation date, with complete nested creator and feature information.

After

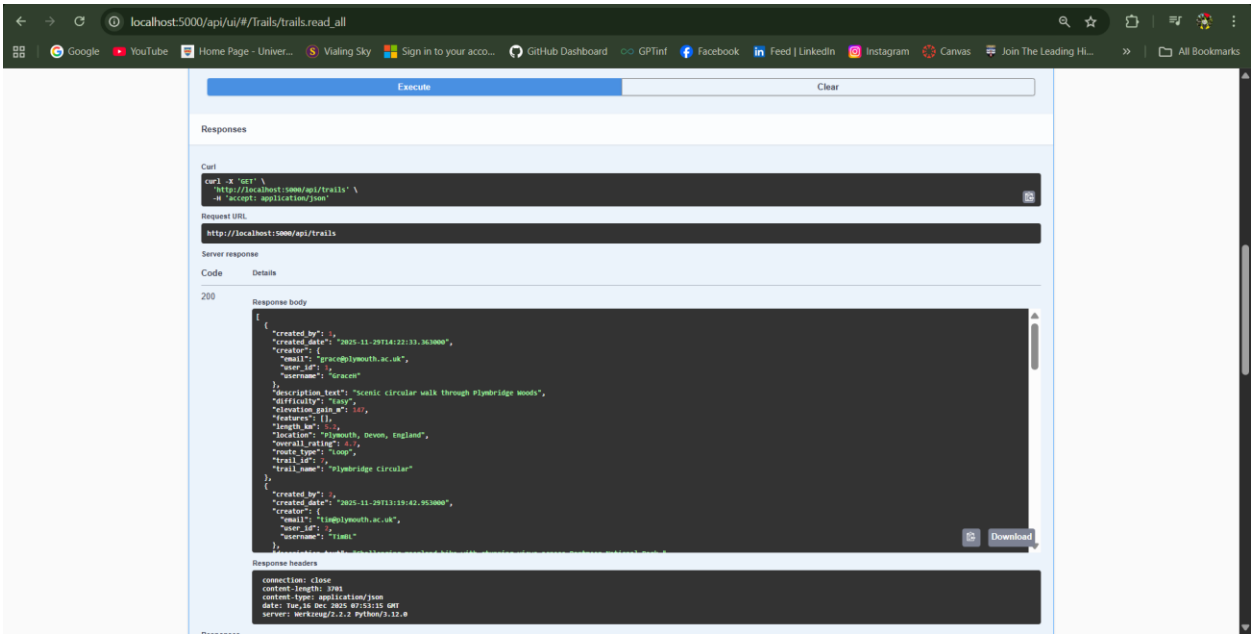

Figure 6 : TC-A1(c)

- HTTP **200 OK** returned
- Response contains an array of trail objects
- Each trail includes core attributes, creator details, and associated features

TC-A2: Create New Trail with Valid Data

Endpoint: `POST /api/trails`

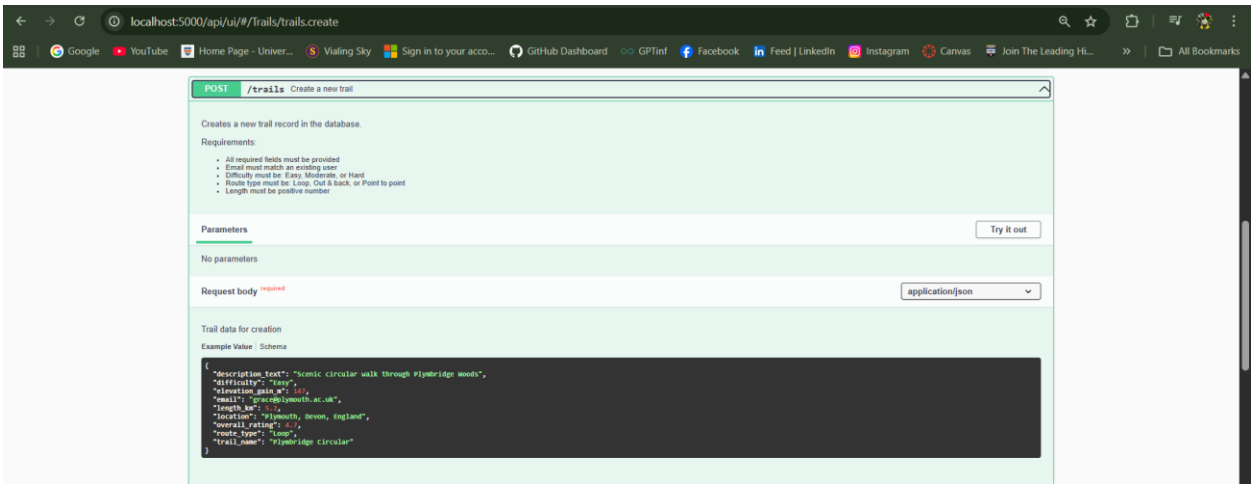

Figure 7 : TC-A2(a)

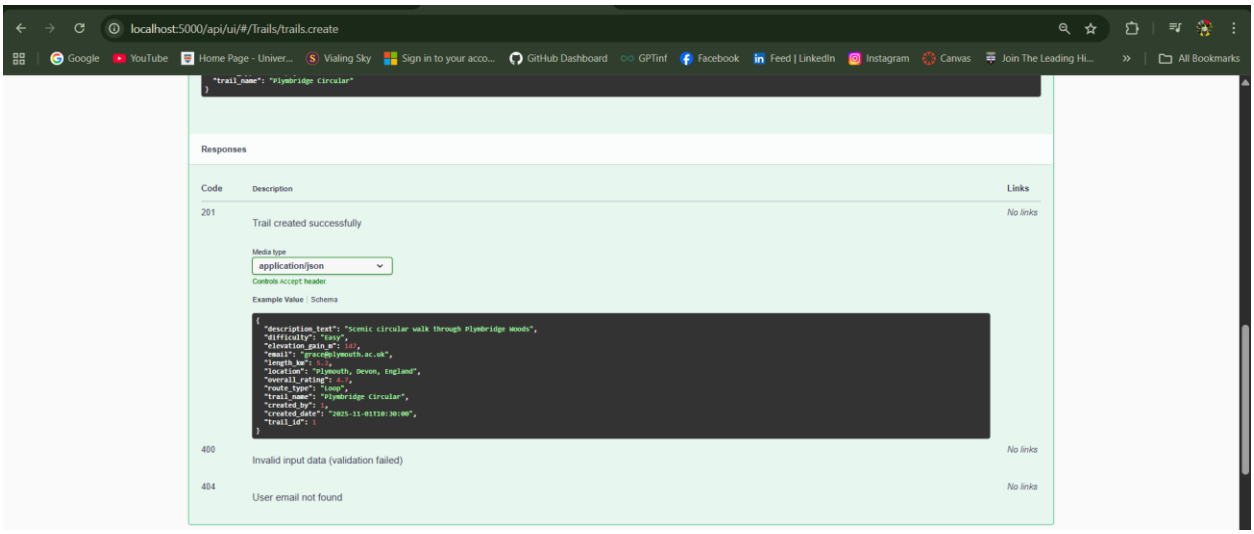

Figure 8 : TC-A2(b)

Purpose

To confirm that a new trail can be created when all required fields are valid and the user email exists.

Before

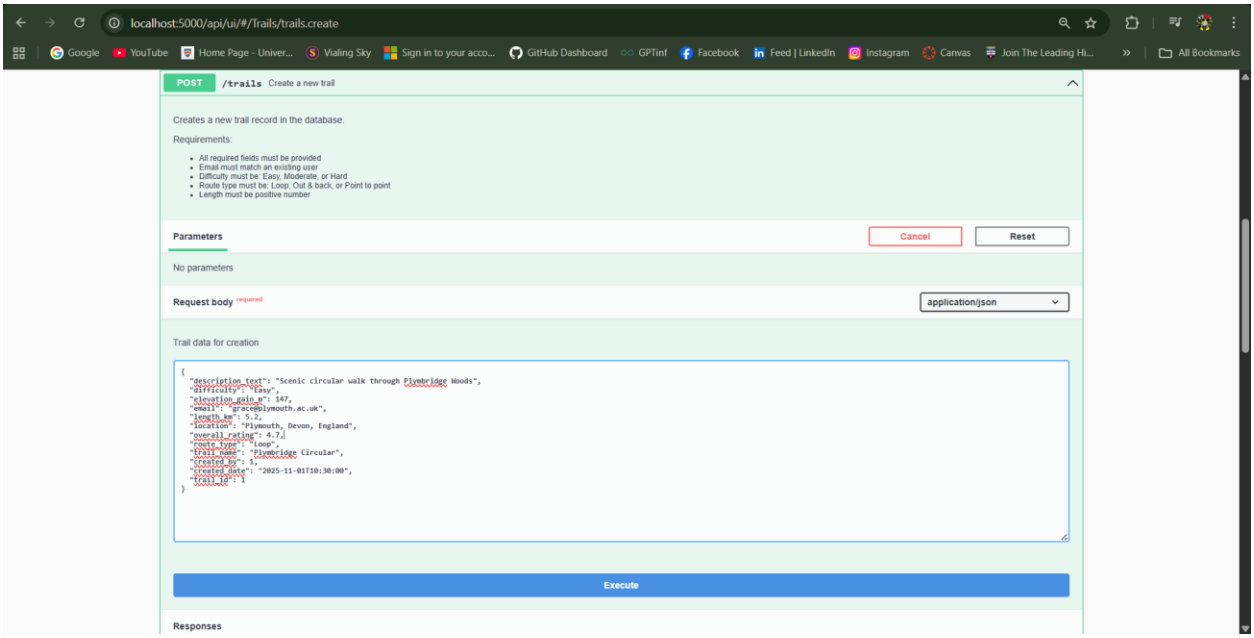

Figure 9 : TC-A2(c)

- User email exists in USER table

After

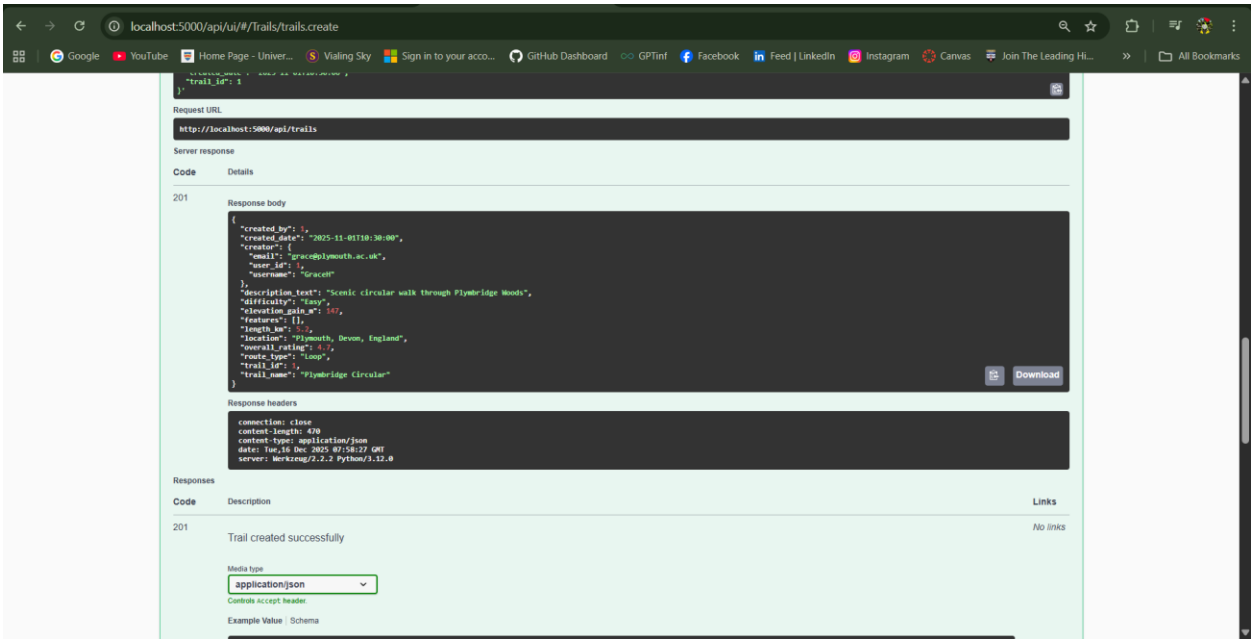

Figure 10 : TC-A2(c)

- HTTP **201 Created**
- New trail_id generated
- Trail persisted in database

TC-A3: Retrieve Specific Trail by ID

Endpoint: `GET /api/trails/{trail_id}`

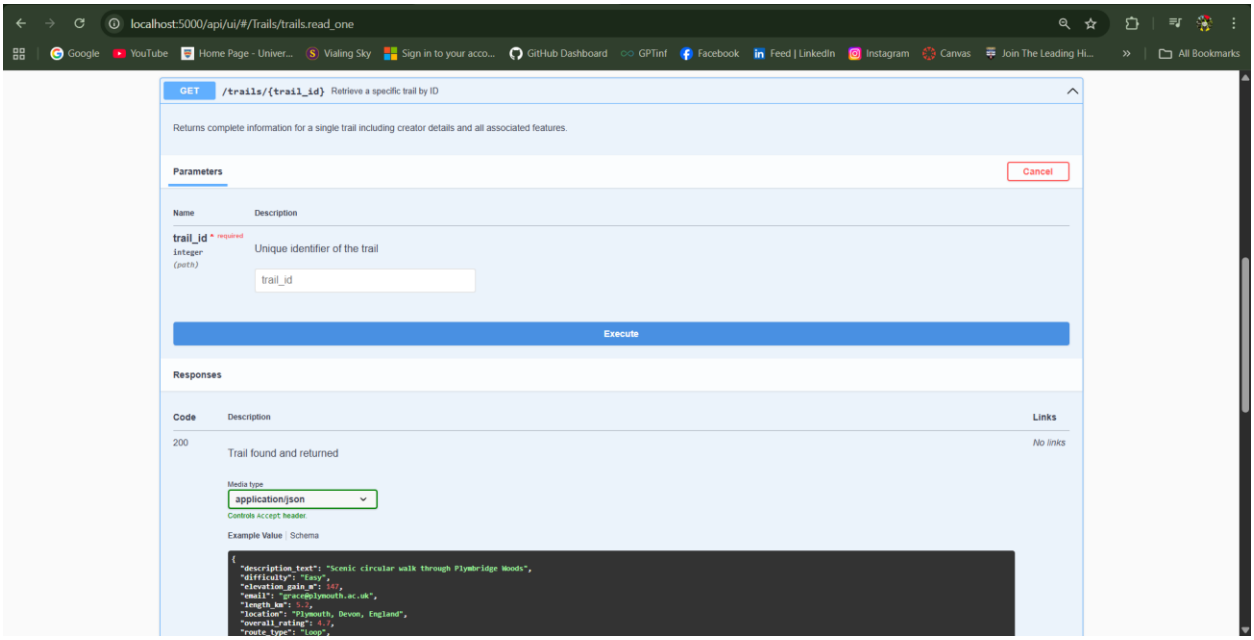

Figure 11 : TC-A3(a)

Purpose

To ensure a single trail can be retrieved using its unique identifier, including all related data.

After

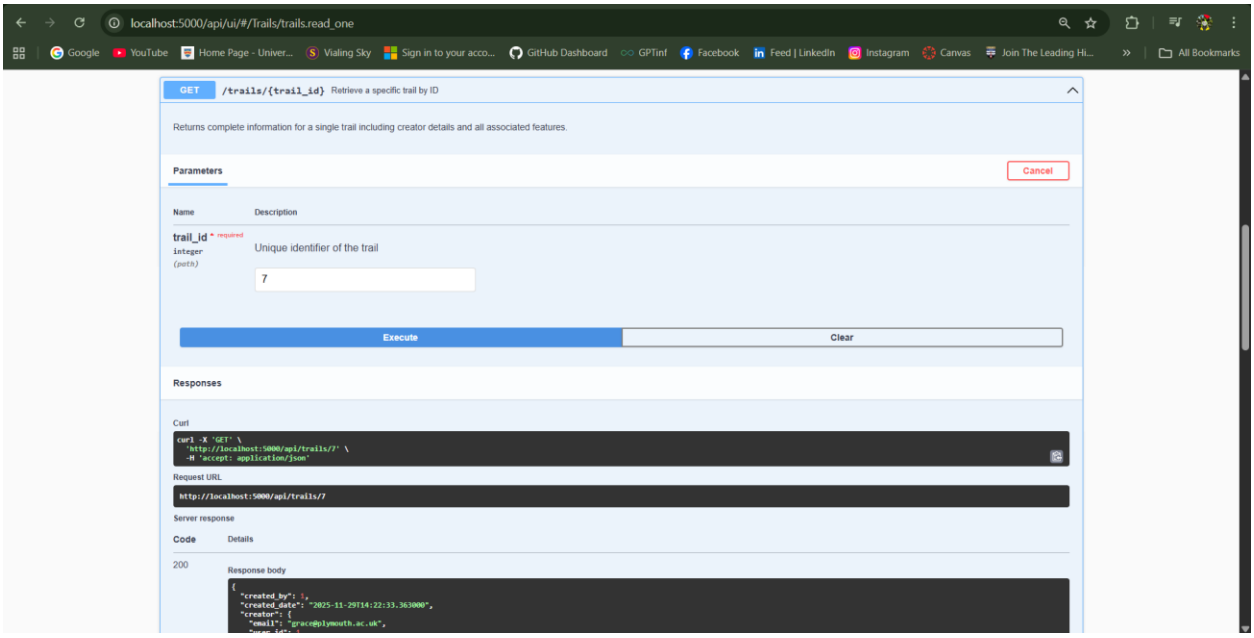

Figure 12 : TC-A3(b)

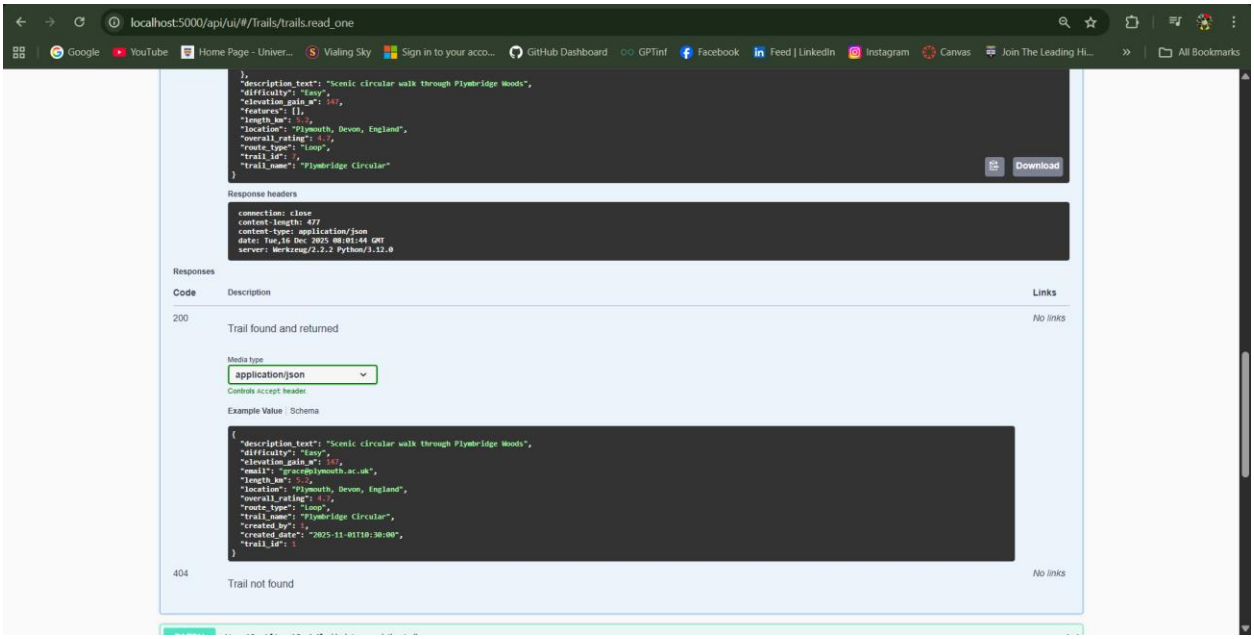

Figure 13 : TC-A3(c)

- HTTP **200 OK**
- Input trail object : 7 returned correctly

TC-A4: Update Trail with Partial Data

Endpoint: `PATCH /api/trails/{trail_id}`

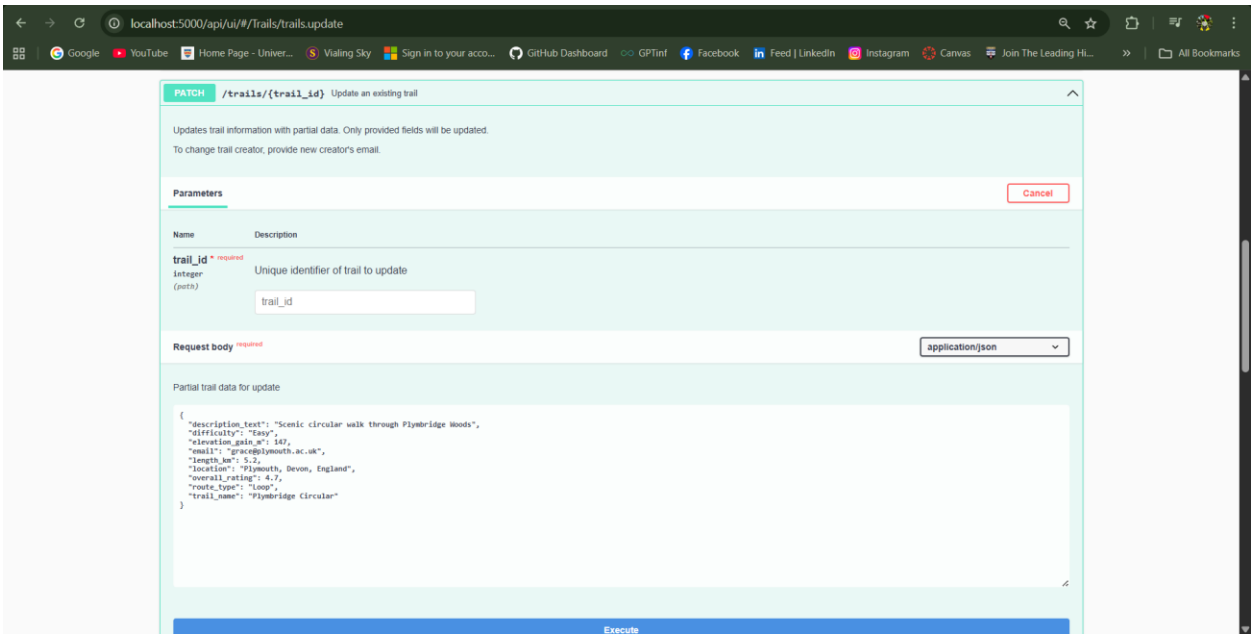

Figure 14 : TC-A4(a)

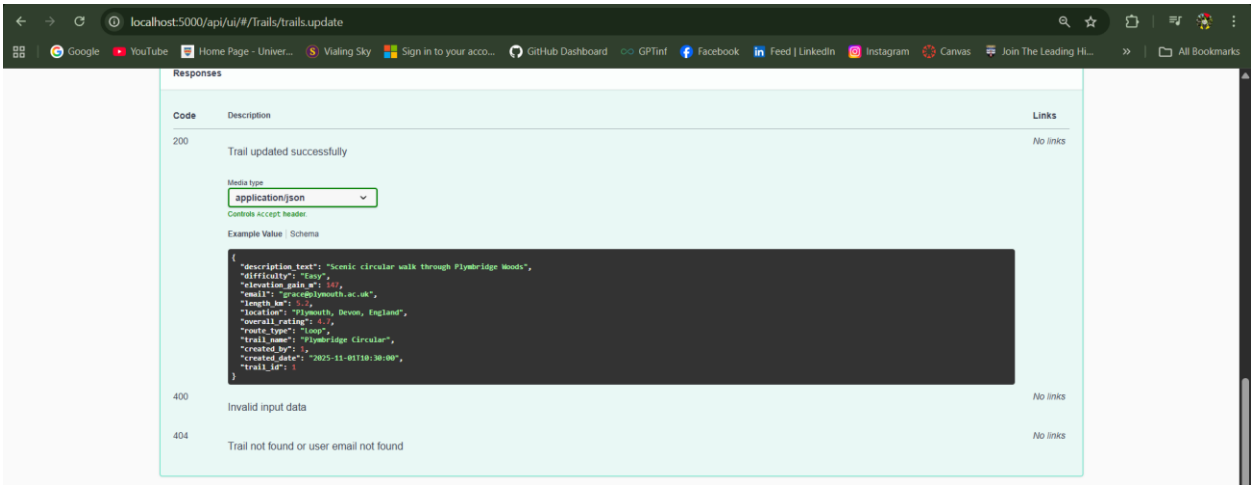

Figure 15 : TC-A4(b)

Purpose

To validate PATCH semantics where only provided fields are updated without affecting other attributes.

Before

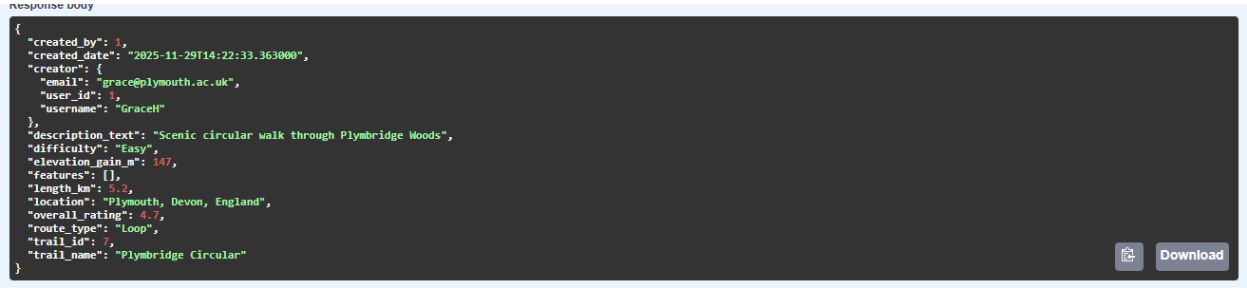

Figure 16 : TC-A4(d)

- Trail exists with original values

After

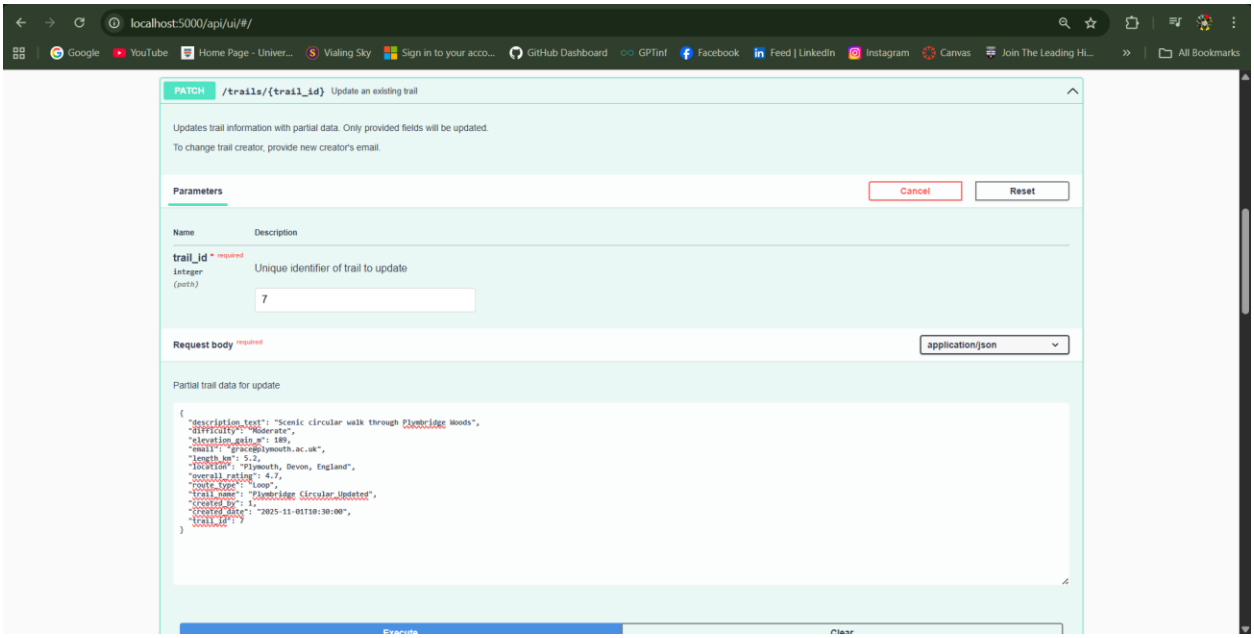

Figure 17 : TC-A4(d)

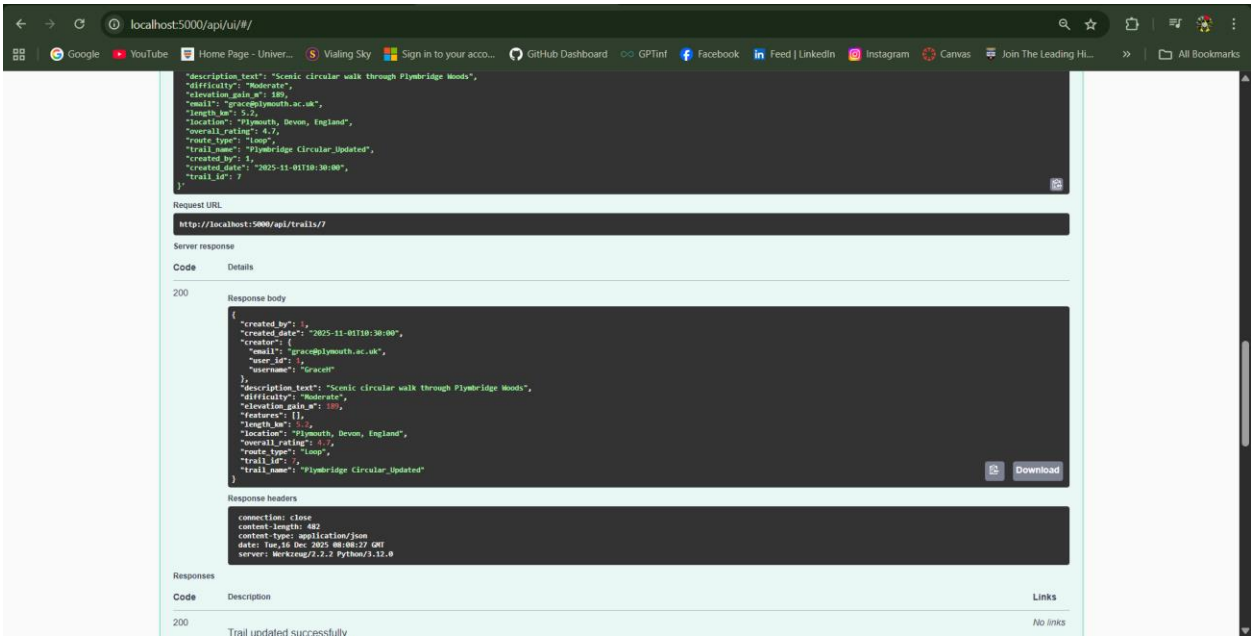

Figure 18 : TC-A4(e)

- HTTP **200 OK**
- Only specified fields updated like difficulty and trail_name.

TC-A5: Delete Existing Trail

Endpoint: `DELETE /api/trails/{trail_id}`

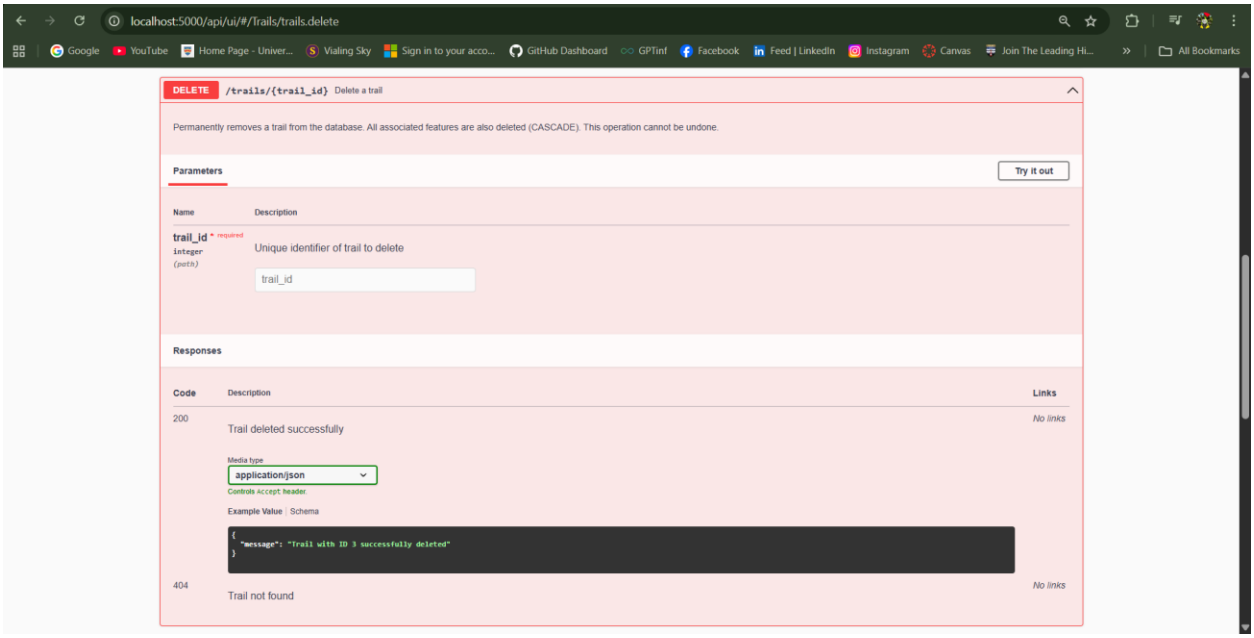

Figure 19 : TC-A5(a)

Purpose

To confirm permanent deletion of a trail and cascade removal of associated features.

Before

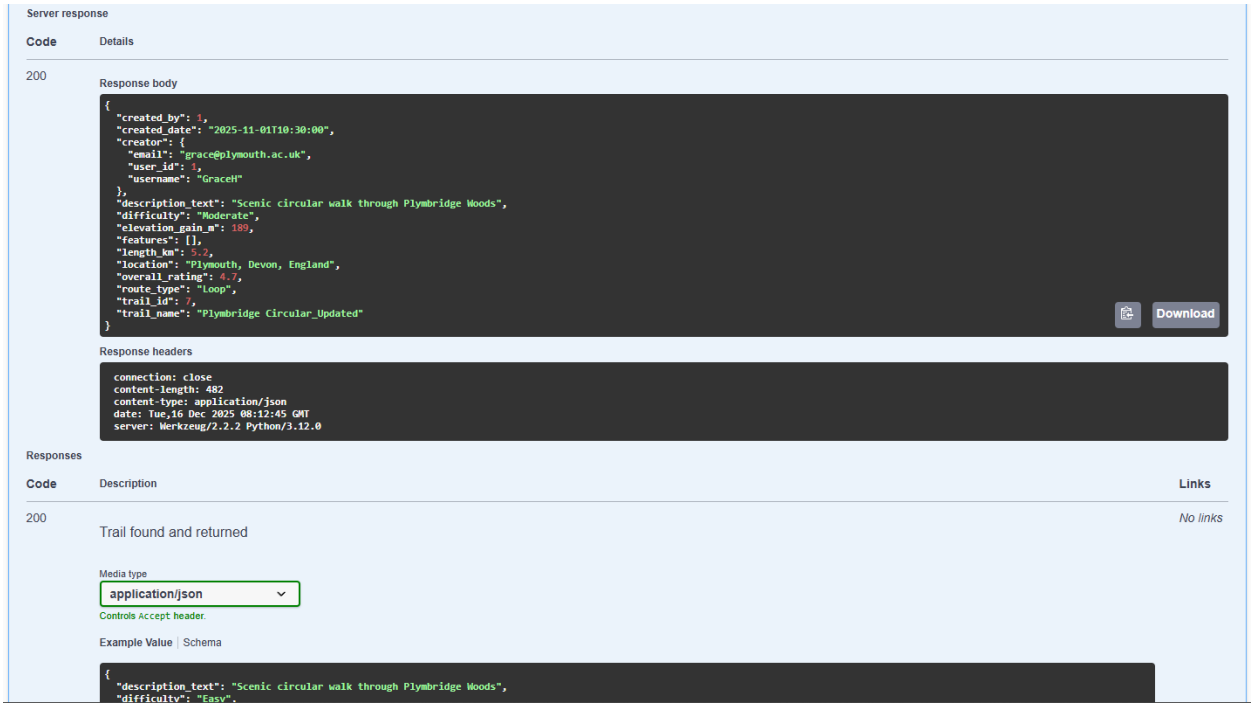

Figure 20 : TC-A5(b)

- Trail and feature records exist

After

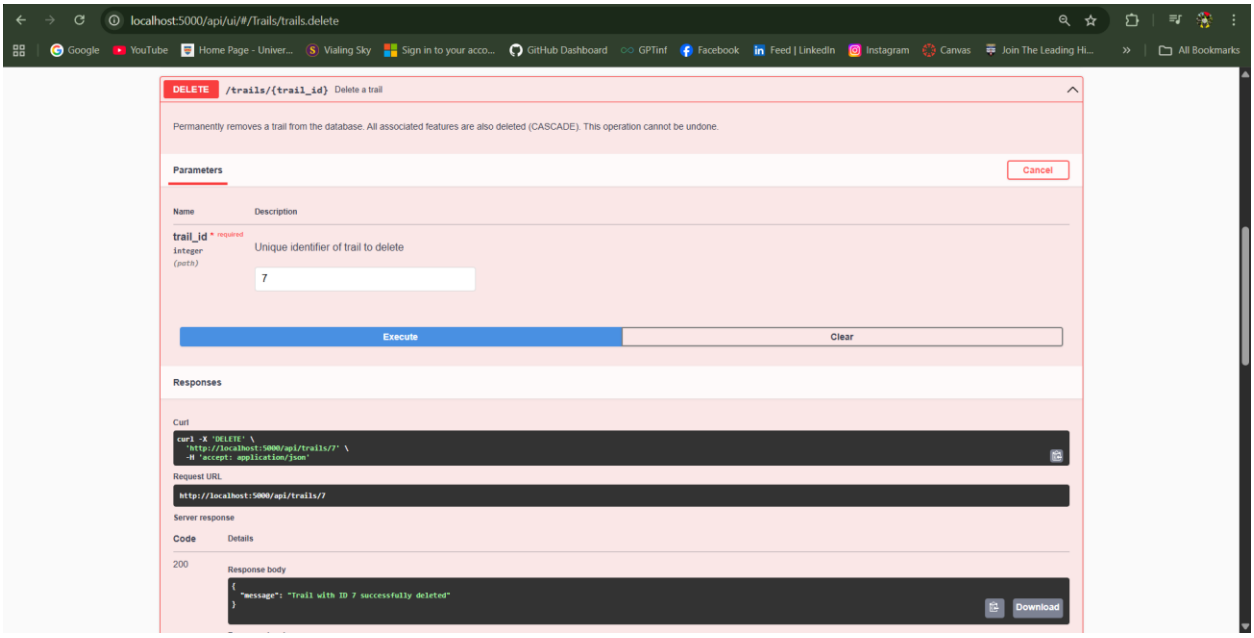

Figure 21 : TC-A5(c)

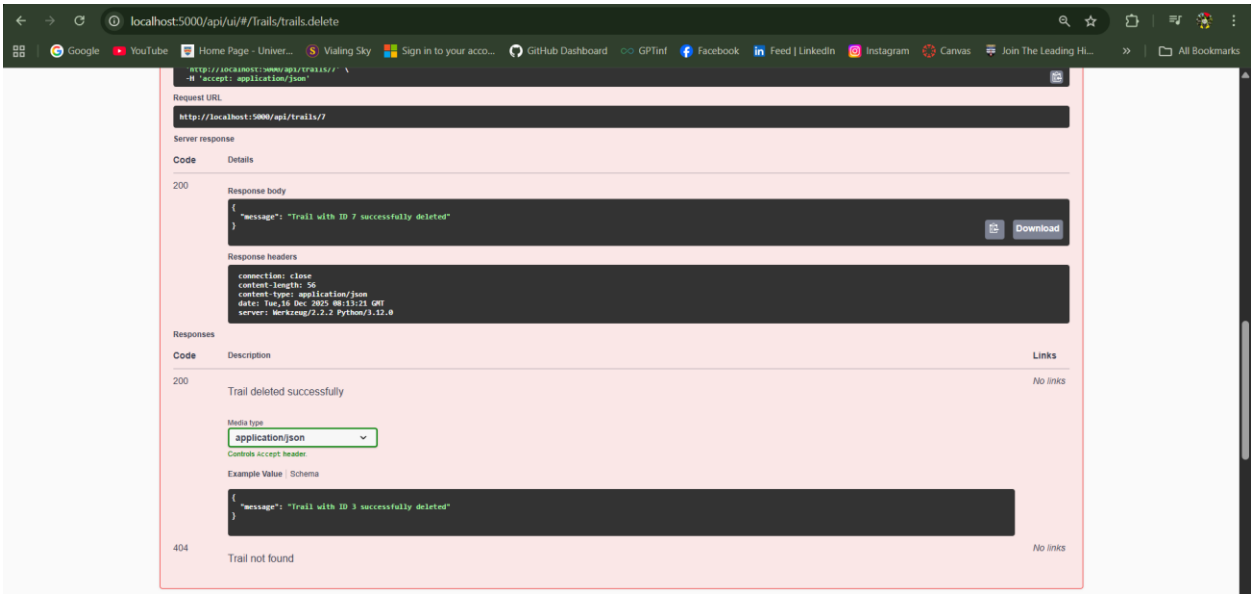

Figure 22 : TC-A5(d)

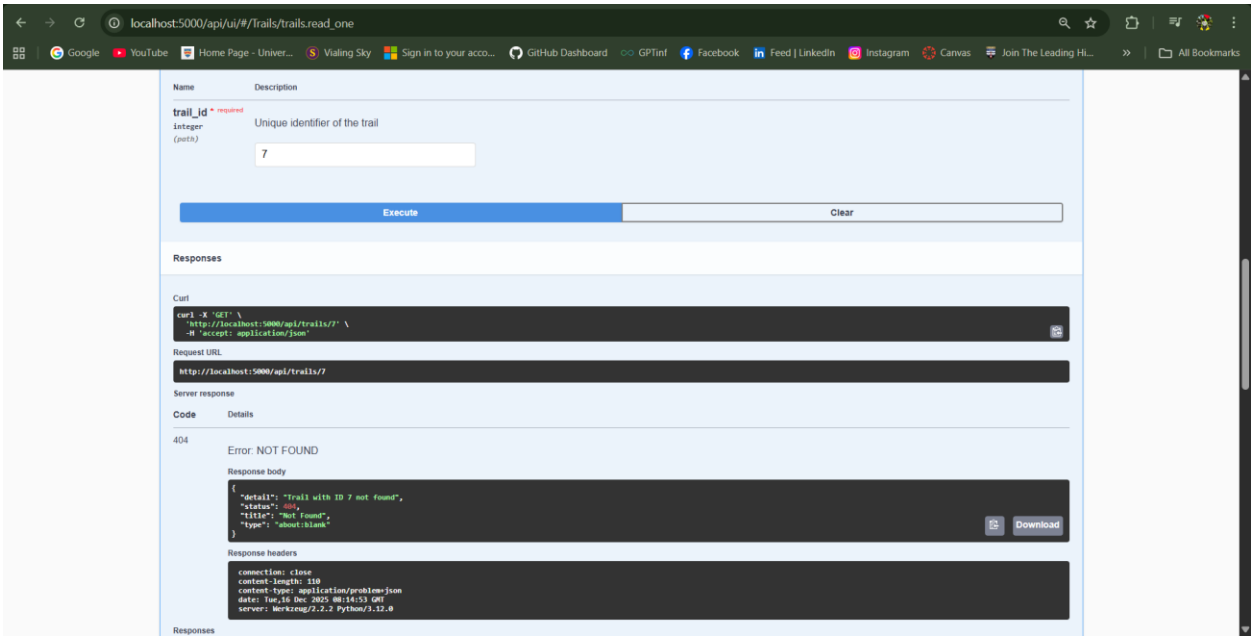

Figure 23 : TC-A5(e)

- HTTP **200 OK**
- Subsequent retrieval returns **404 Not Found**

TC-A6: Retrieve Features for Trail

Endpoint: `GET /api/trails/{trail_id}/features`

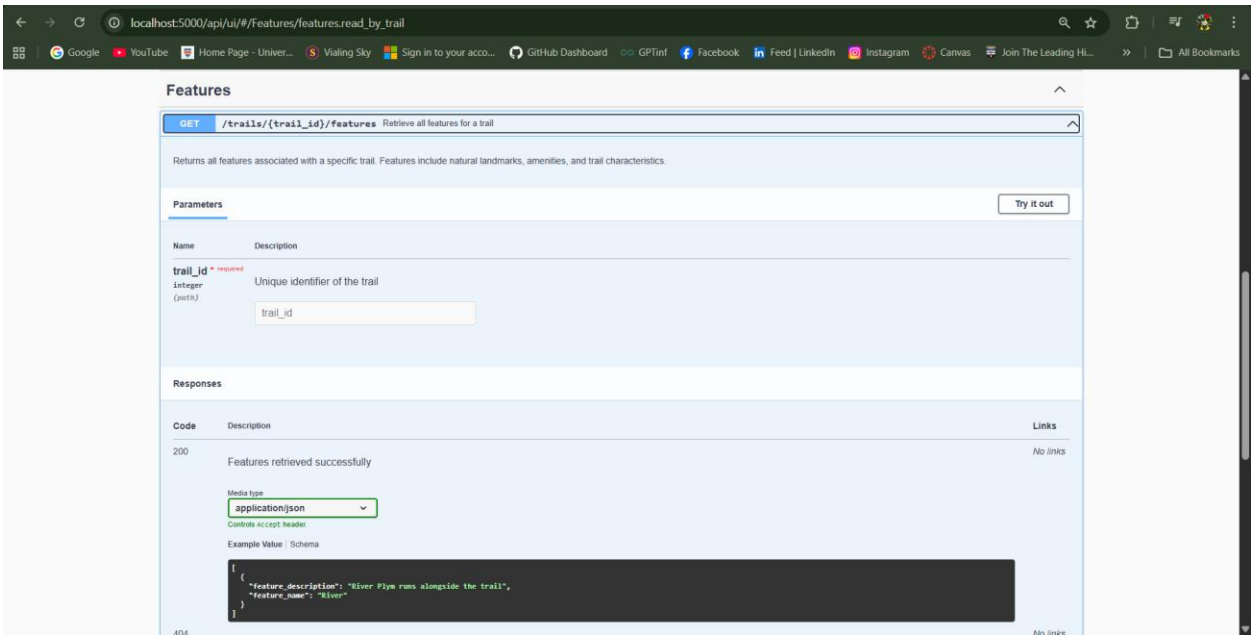

Figure 24 : TC-A6(a)

Purpose

To verify that all features associated with a trail are correctly retrieved via the junction table.

After

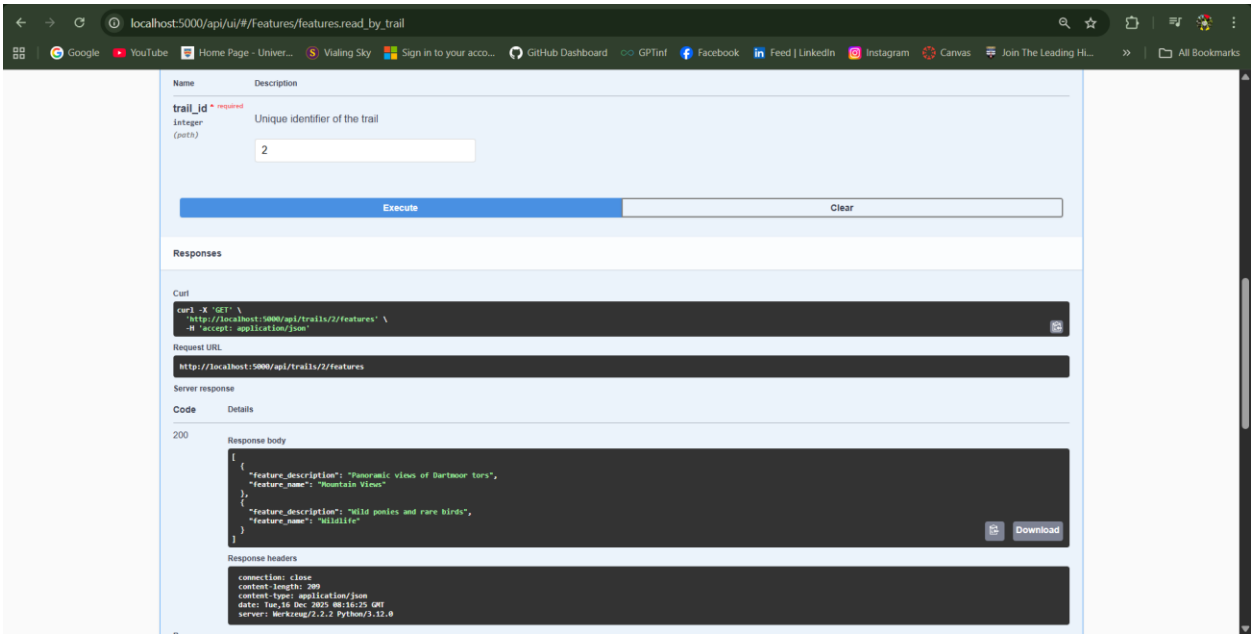

Figure 25 : TC-A6(b)

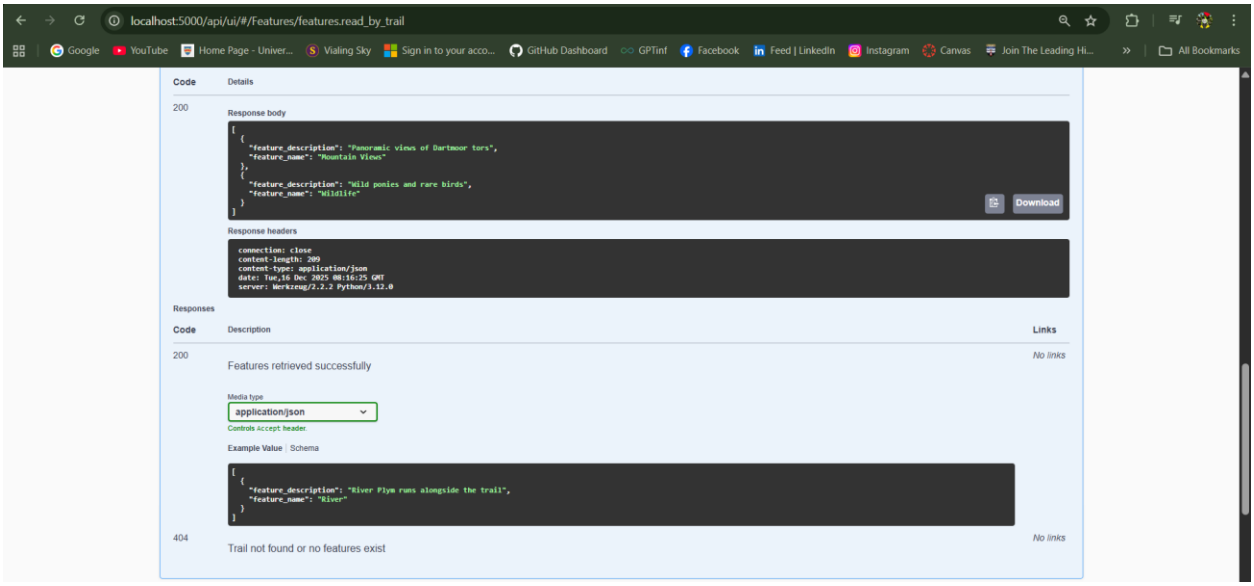

Figure 26 : TC-A6(c)

- HTTP **200 OK**
- Feature array of trail_id :2 returned correctly.

TC-A7: Retrieve Trails from Database View

Endpoint: `GET /api/trails/view`

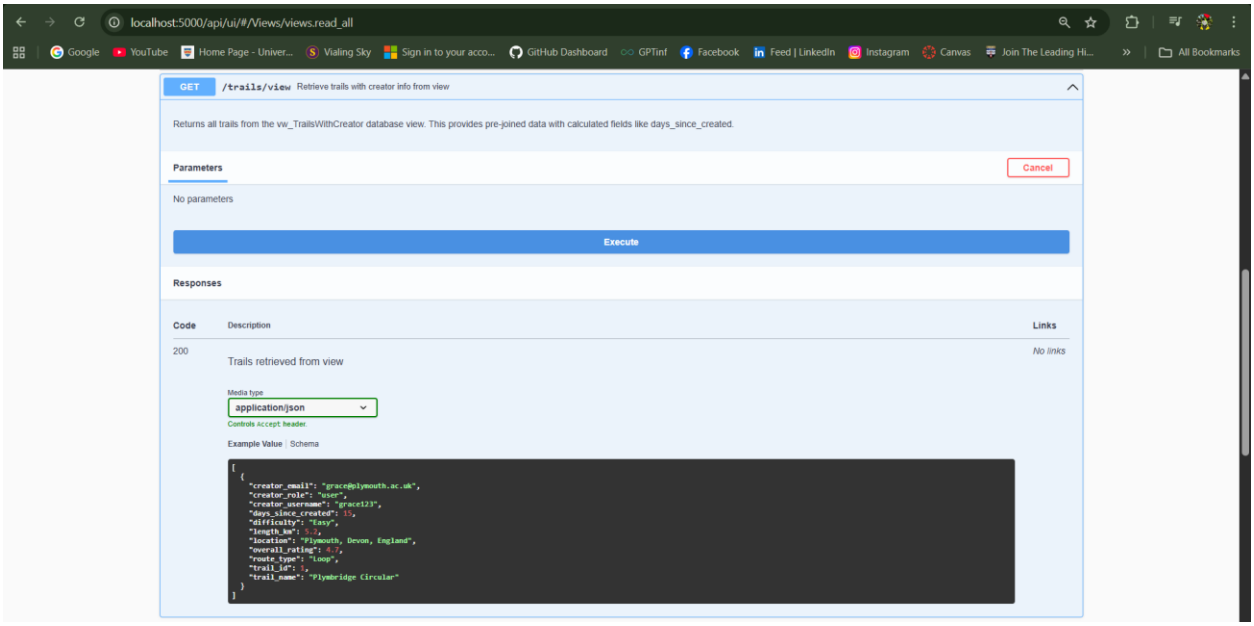

Figure 27 : TC-A7(a)

Purpose

To validate correct exposure of the vw_TrailsWithCreator database view via the API.

After

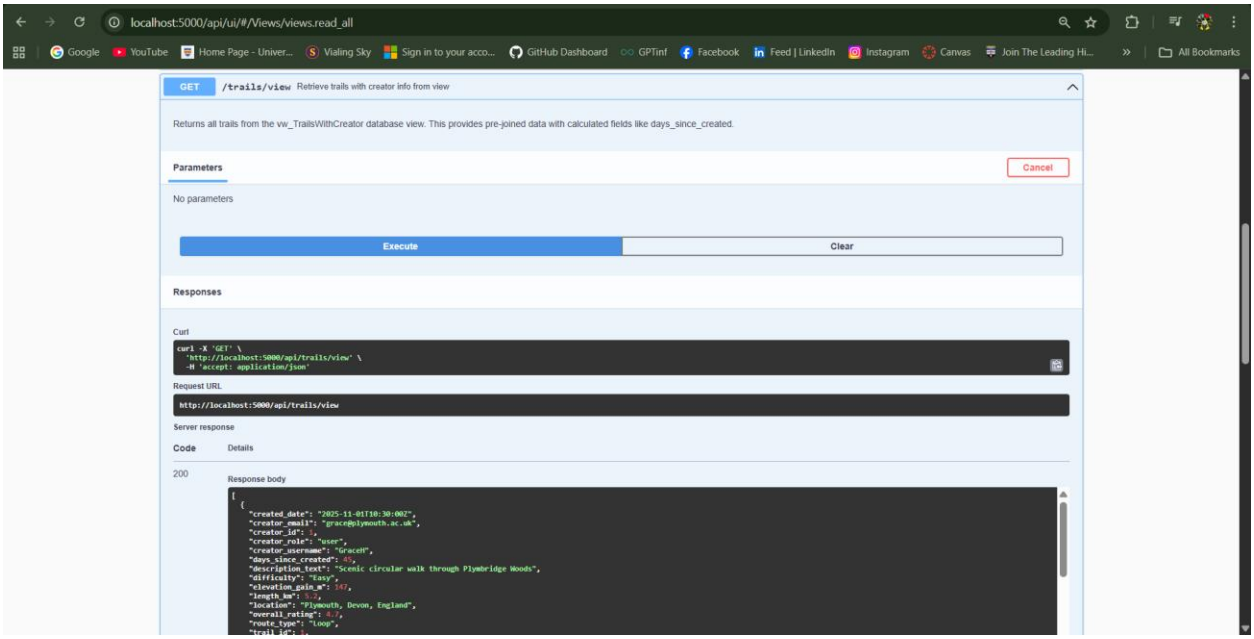

Figure 28 : TC-A7(b)

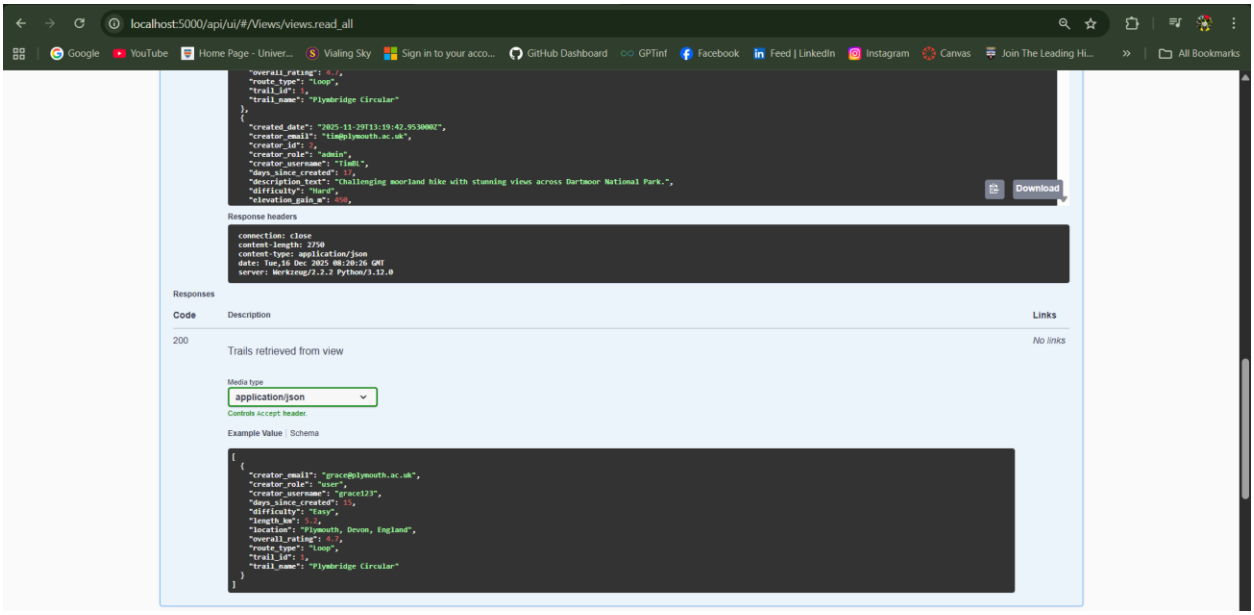

Figure 29 : TC-A7(c)

- HTTP **200 OK**
- Flattened trail and creator data returned
- Calculated fields included

TC-A8: Retrieve Single Trail from Database View

Endpoint: `GET /api/trails/view/{trail_id}`

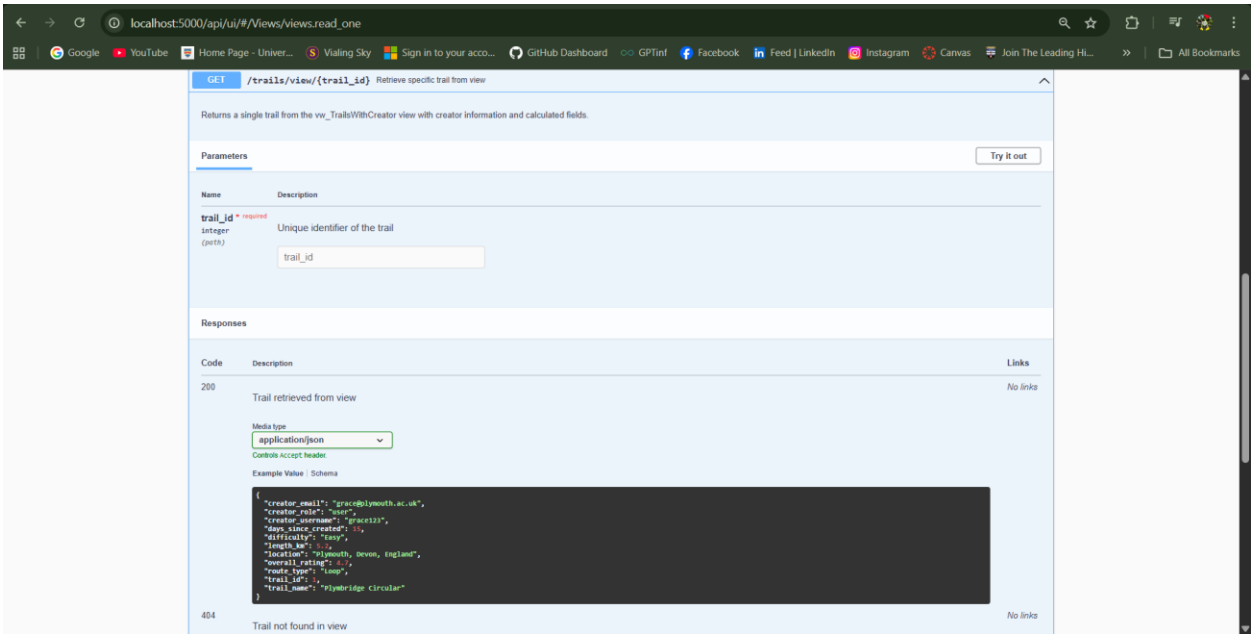

Figure 30 : TC-A8(a)

Purpose

To confirm retrieval of an individual trail from the database view.

After

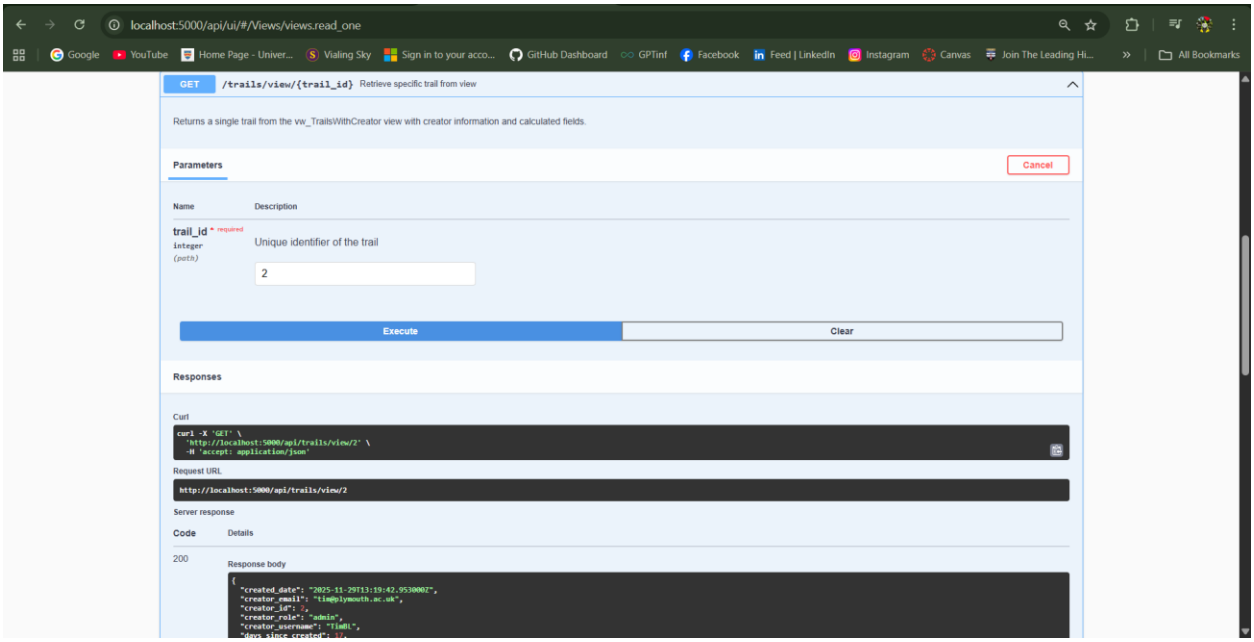

Figure 31 : TC-A8(b)

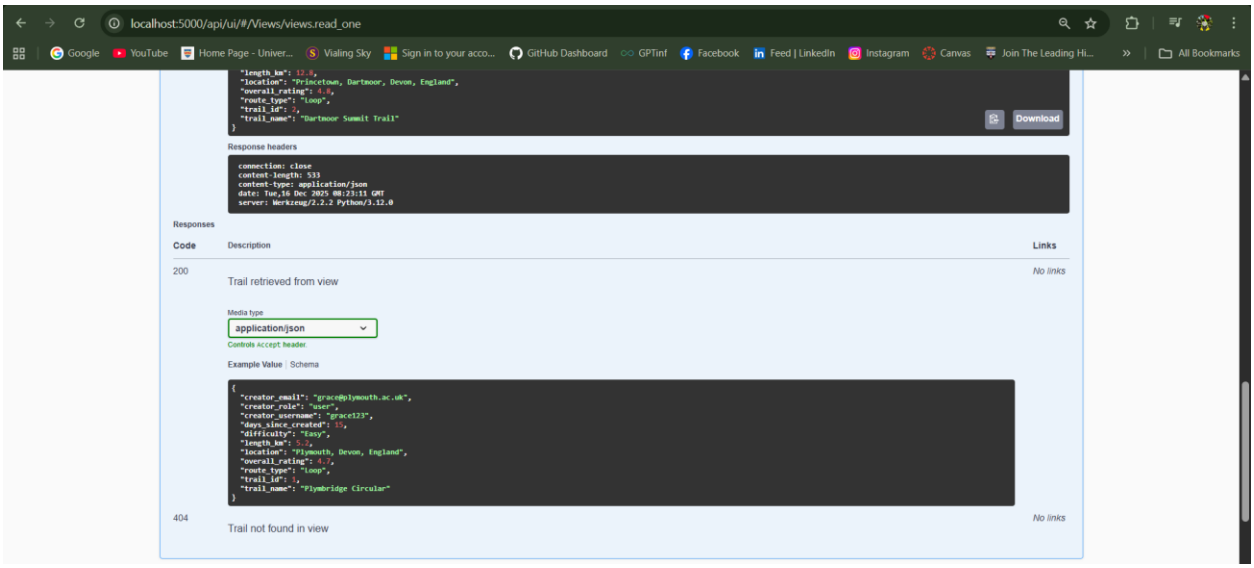

Figure 32 : TC-A8(c)

- HTTP **200 OK**
- Single flattened view record returned

TC-A9: Retrieve All Users

Endpoint: `GET /api/users`

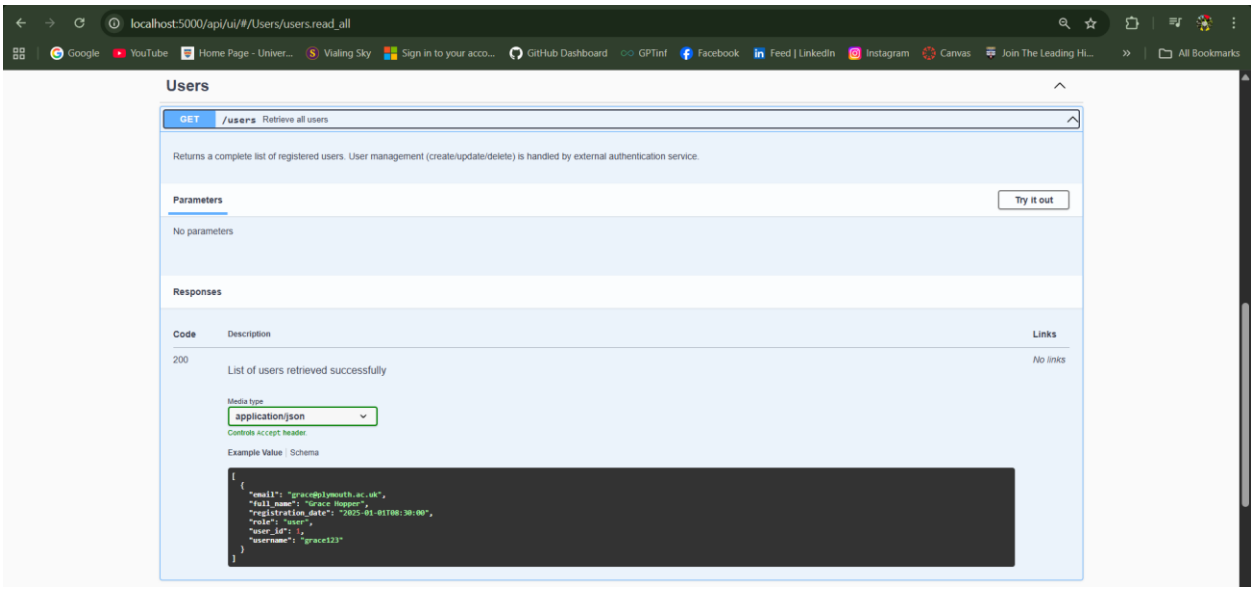

Figure 33 : TC-A9(a)

Purpose

To verify read-only access to registered user accounts.

After

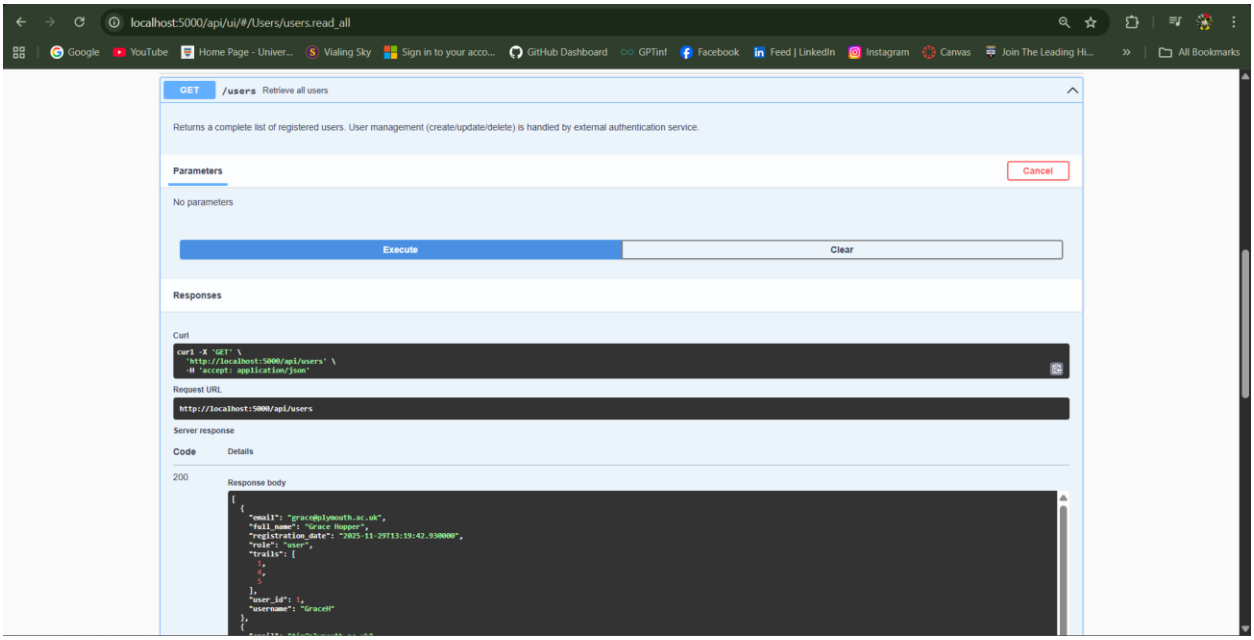

Figure 34 : TC-A9(b)

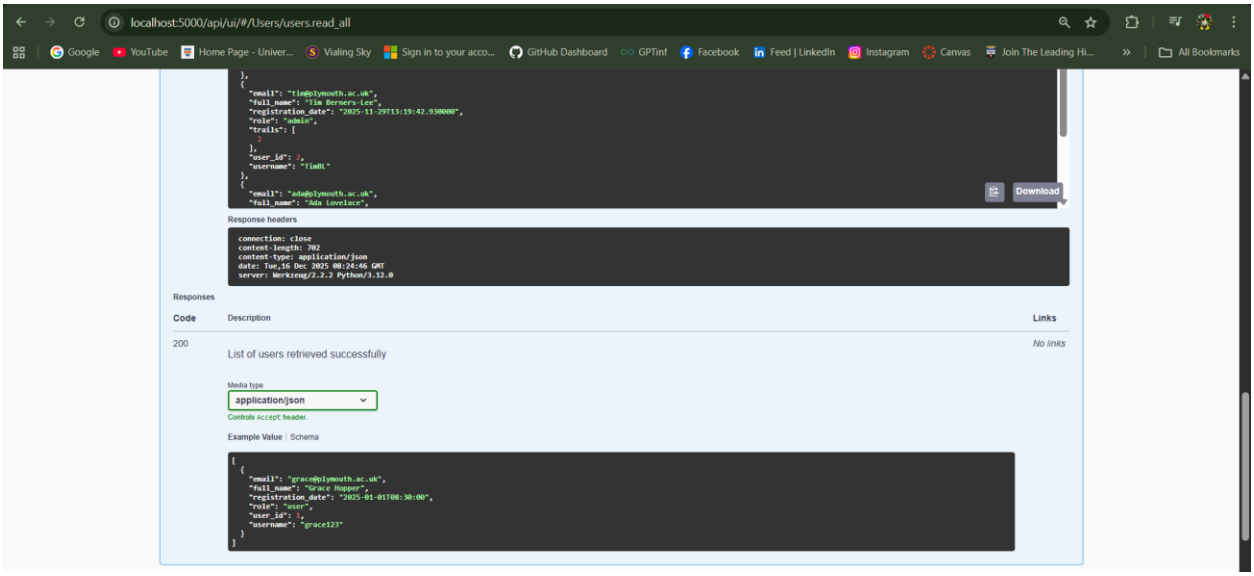

Figure 35 : TC-A9(c)

- HTTP **200 OK**
- Array of user objects returned

TC-A10: Retrieve Specific User by ID

Endpoint: `GET /api/users/{user_id}`

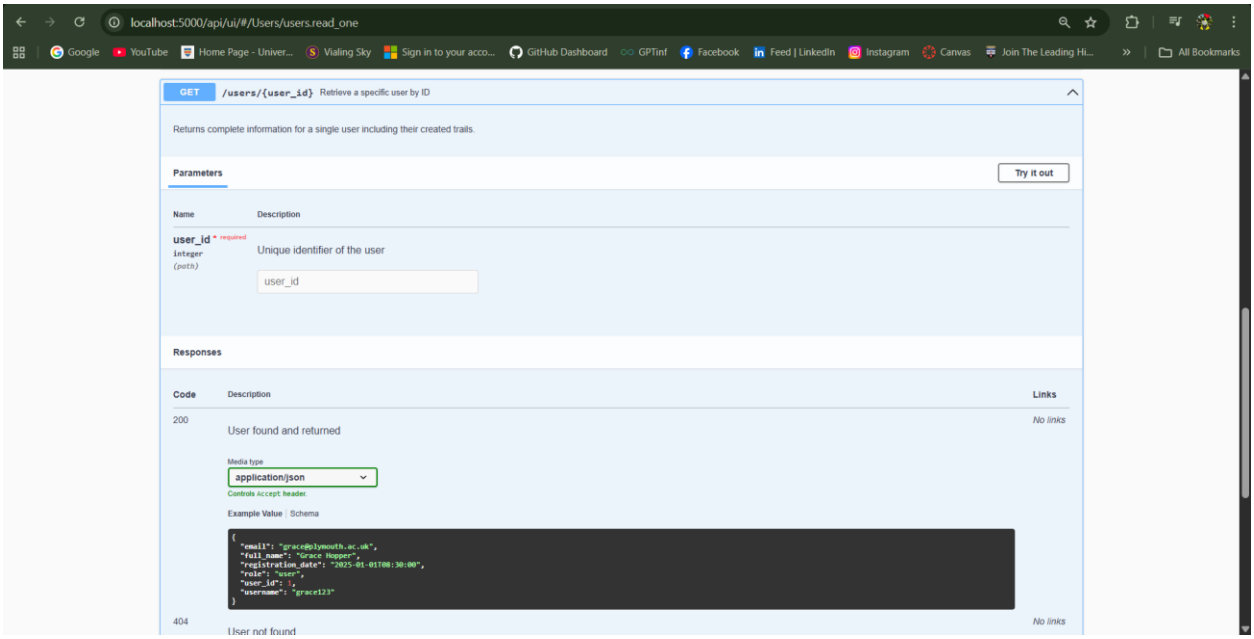

Figure 36 : TC-A10(a)

Purpose

To ensure a single user and their associated trails can be retrieved.

After

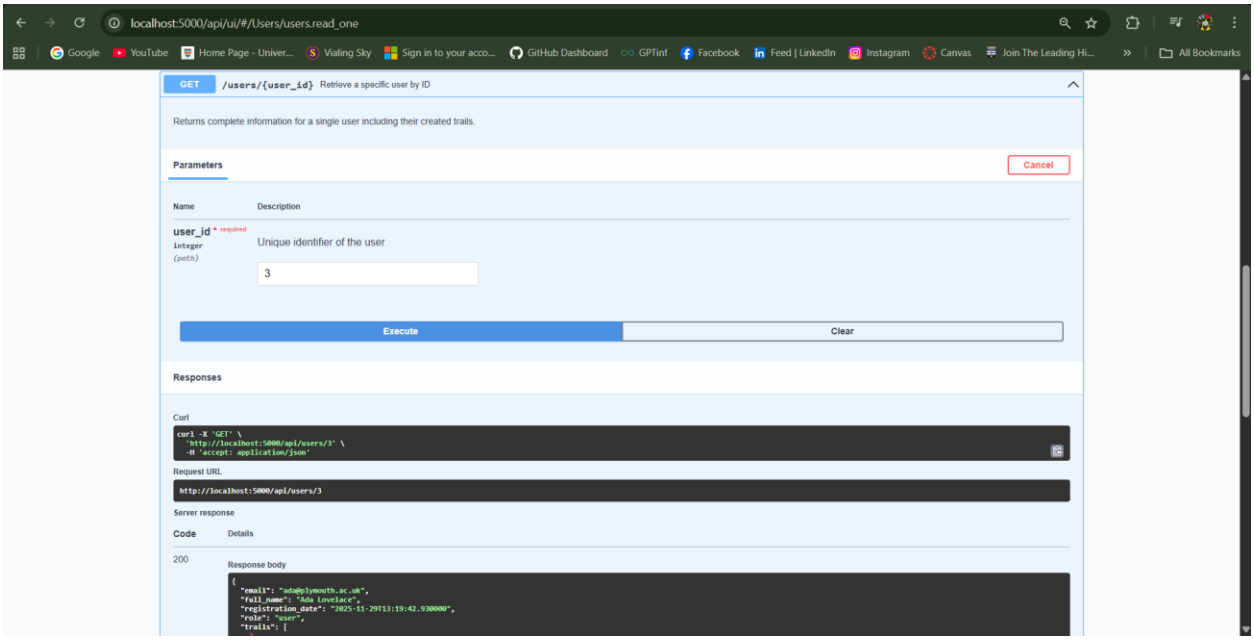

Figure 37 : TC-A10(b)

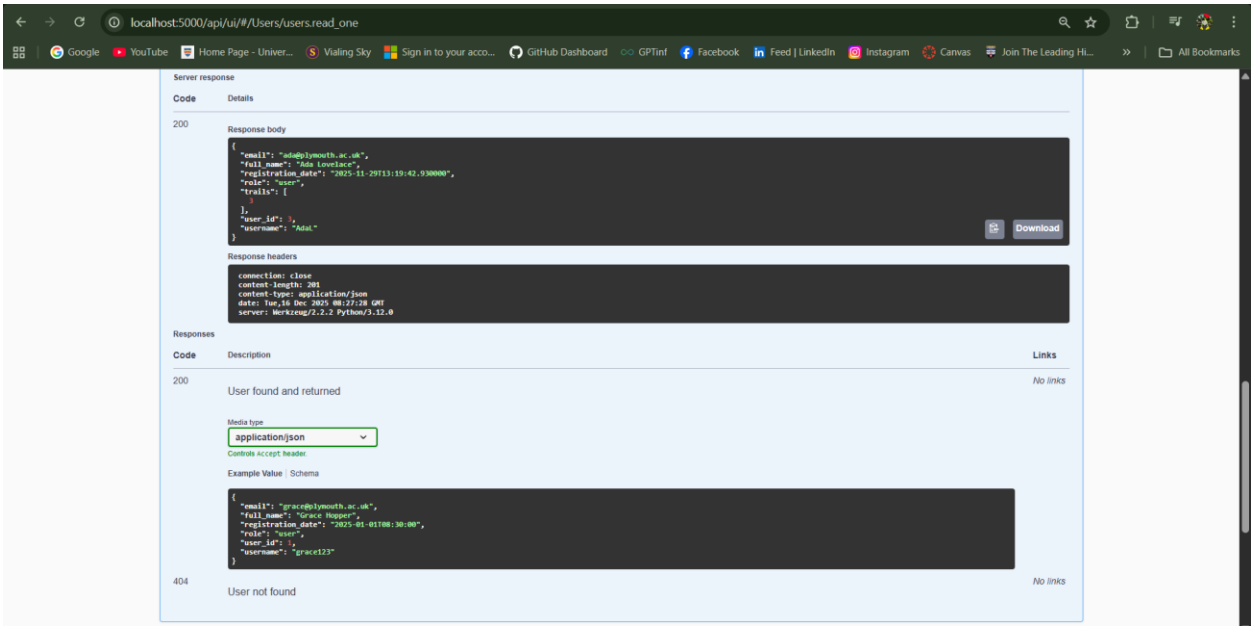

Figure 38 : TC-A10(c)

- HTTP **200 OK**
- User object returned

TC-A11: Retrieve Non-Existent User

Endpoint: `GET /api/users/{user_id}`

Purpose

To validate correct handling of invalid user identifiers.

After

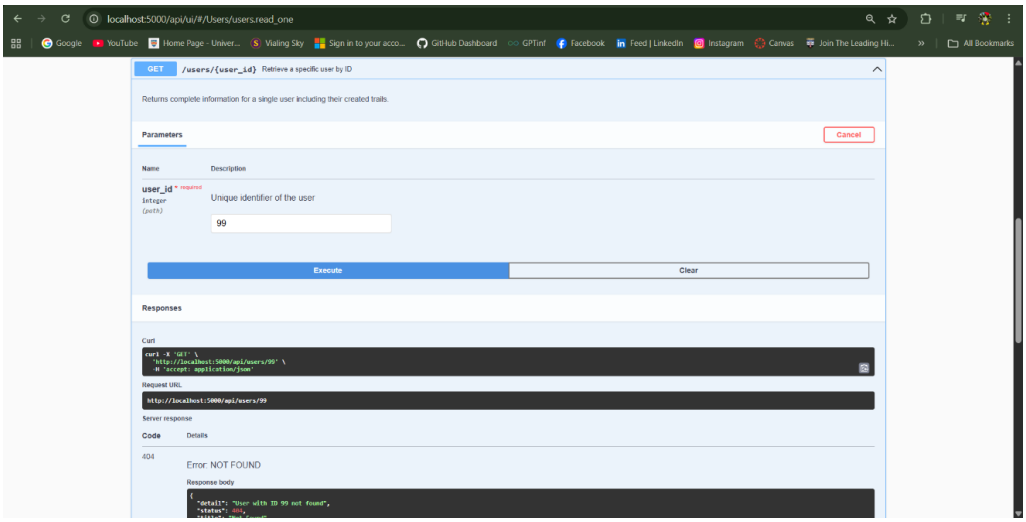

Figure 39 : TC-A11(a)

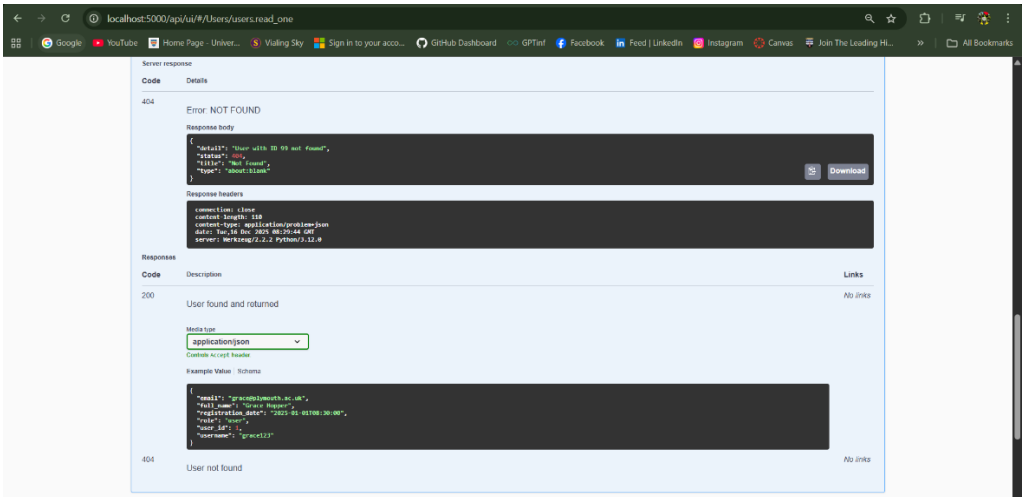

Figure 40 : TC-A11(b)

- HTTP **404 Not Found**
- Input user_id : 99 and Error message returned

TC-A12: Create Trail with Invalid Enum Value

Endpoint: `POST /api/trails`

Purpose

To ensure enum constraints for difficulty are enforced at schema level.

Before

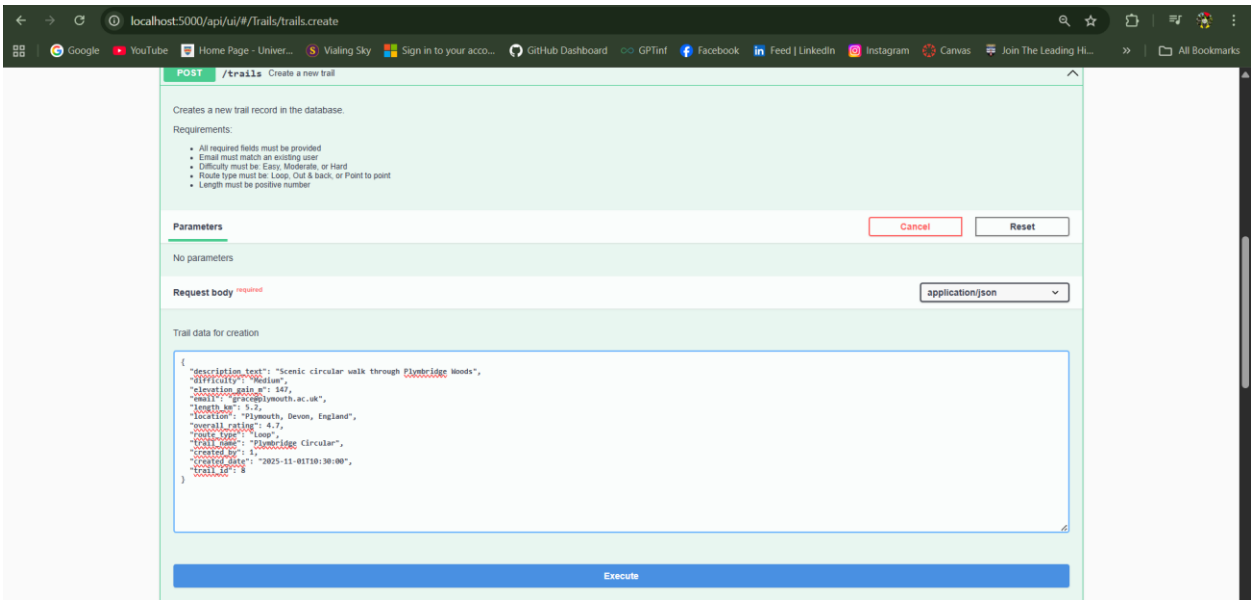

Figure 41 : TC-A12(a)

- Valid enum values defined in OpenAPI

After

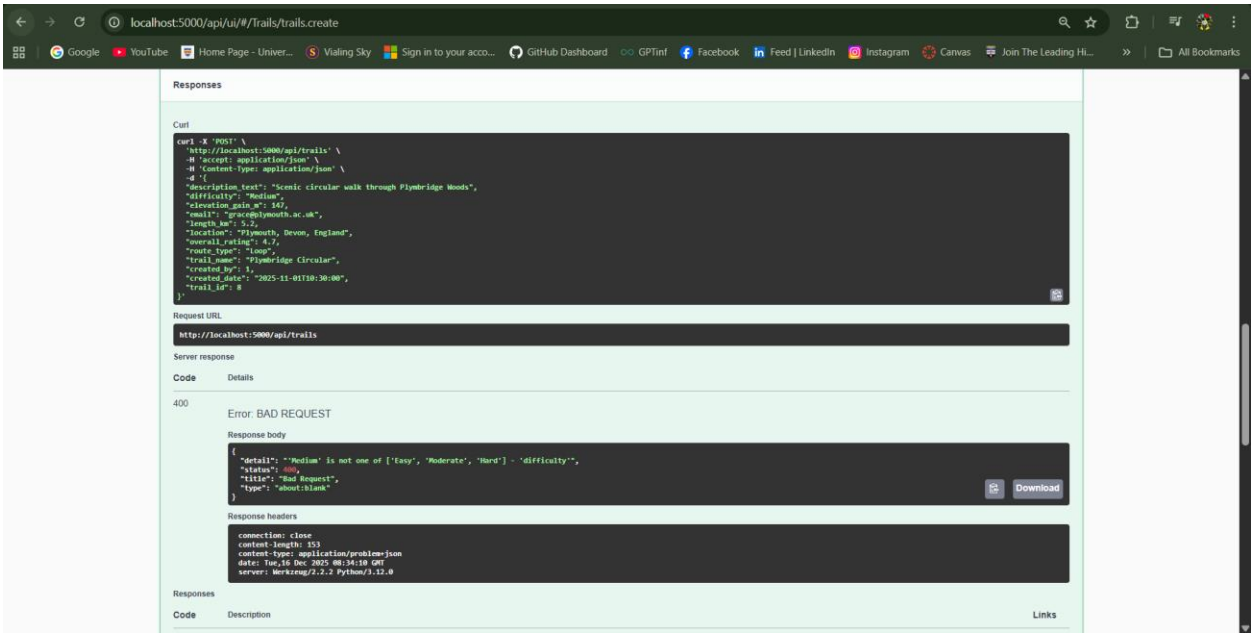

Figure 42 : TC-A12(b)

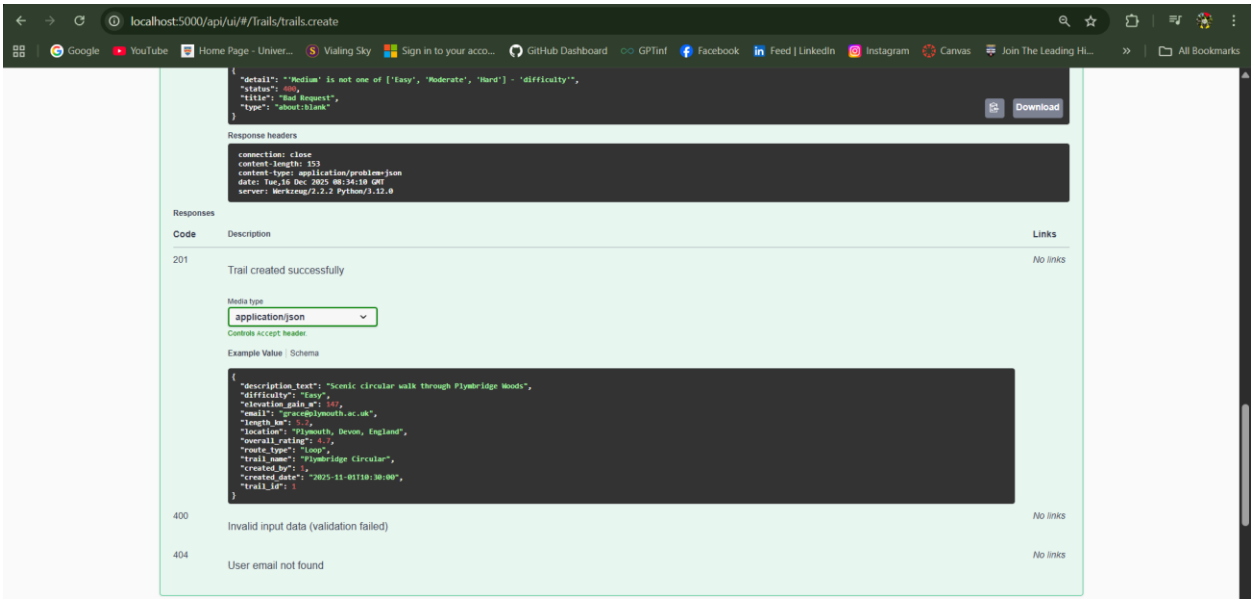

Figure 43 : TC-A12(c)

- HTTP **400 Bad Request**
- Validation error returned
- No trail created

TC-A13: Create Trail with Non-Existent User Email

Endpoint: `POST /api/trails`

Purpose

To confirm referential integrity when linking trails to users.

Before

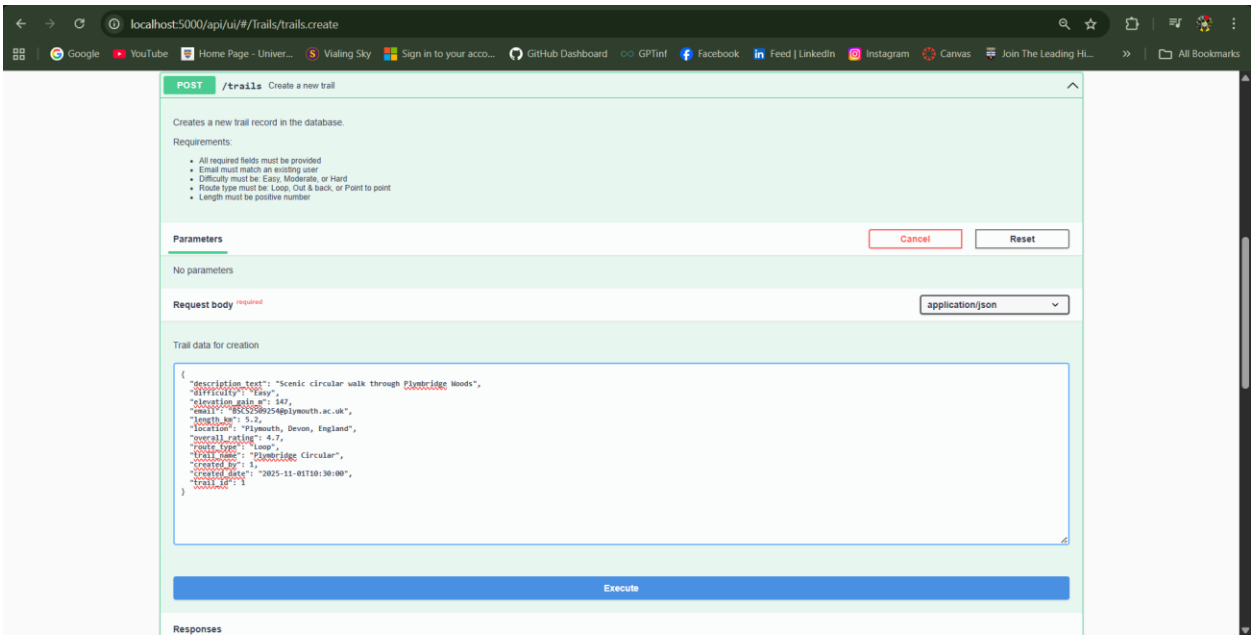

Figure 44 : TC-A13(a)

- Email does not exist in USER table

After

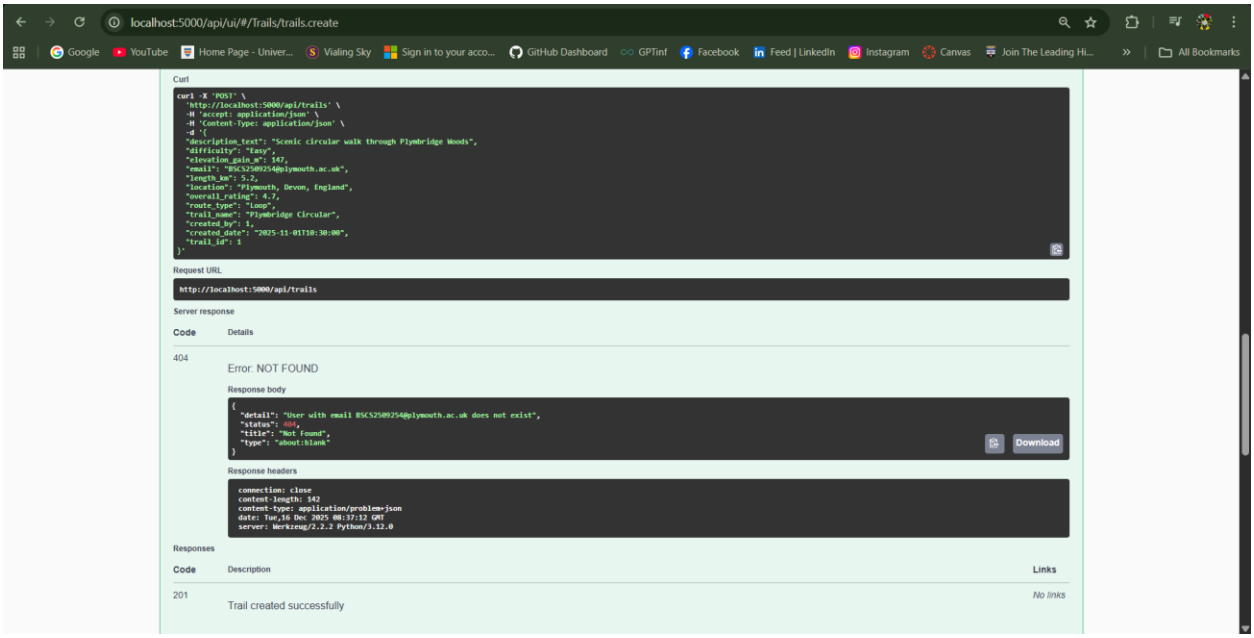

Figure 45 : TC-A13(b)

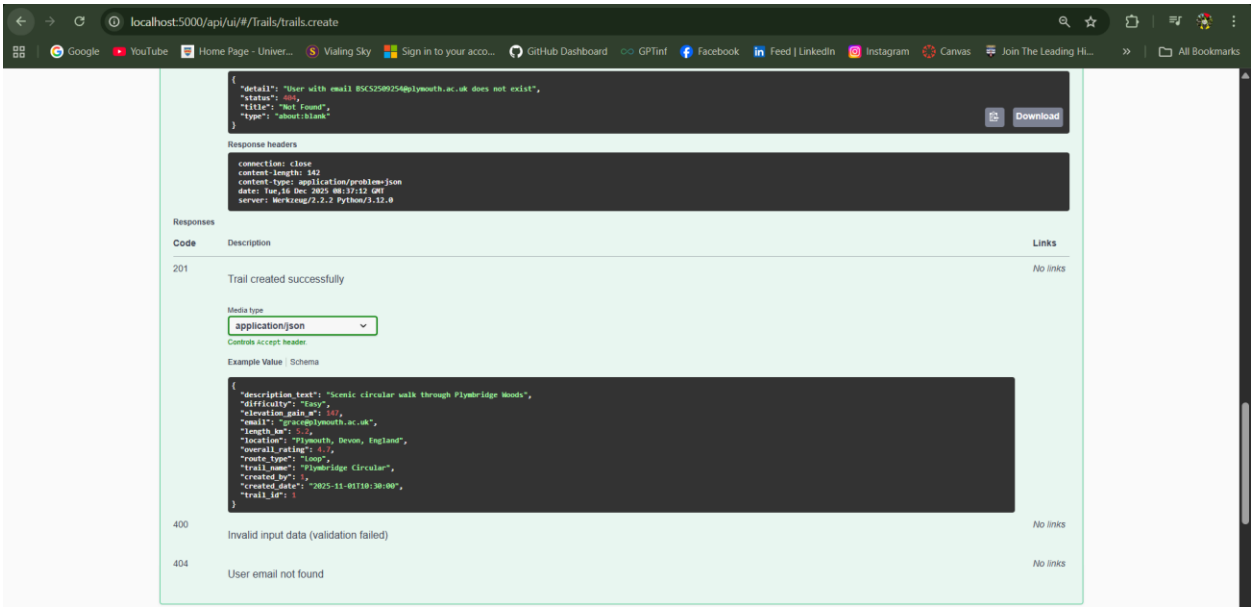

Figure 46 : TC-A13(c)

- HTTP **404 Not Found**
- Trail creation aborted

TC-A14: Update Trail with Invalid Enum Value

Endpoint: `PATCH /api/trails/{trail_id}`

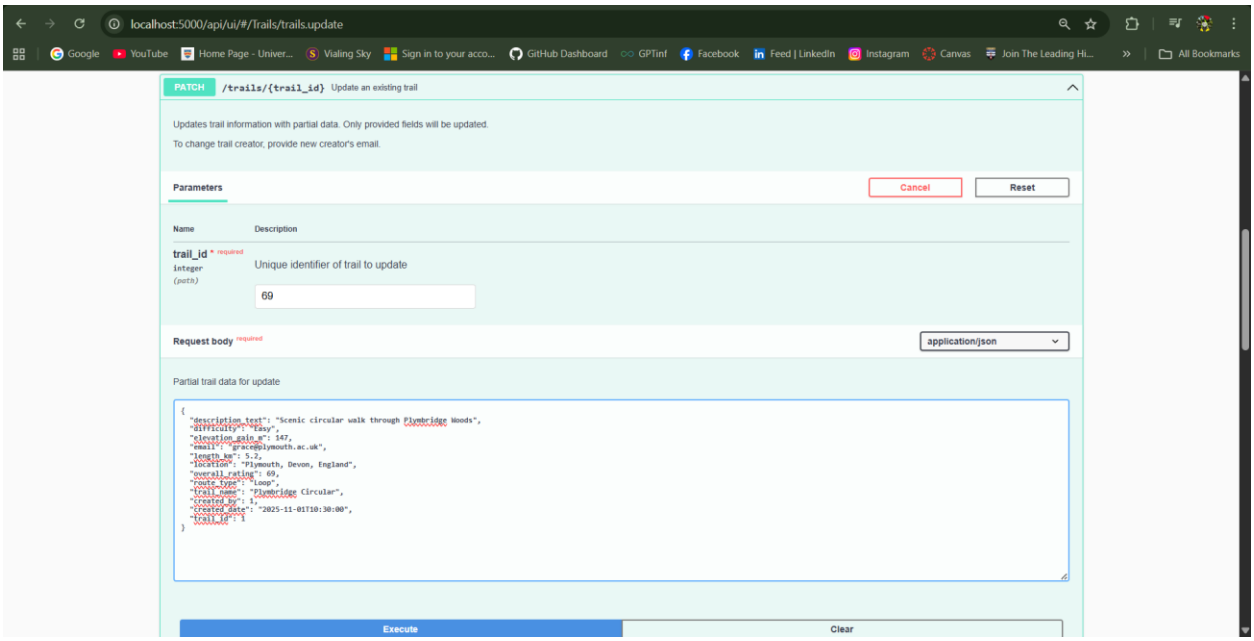

Figure 47 : TC-A14(a)

Purpose

To verify validation enforcement during partial updates.

After

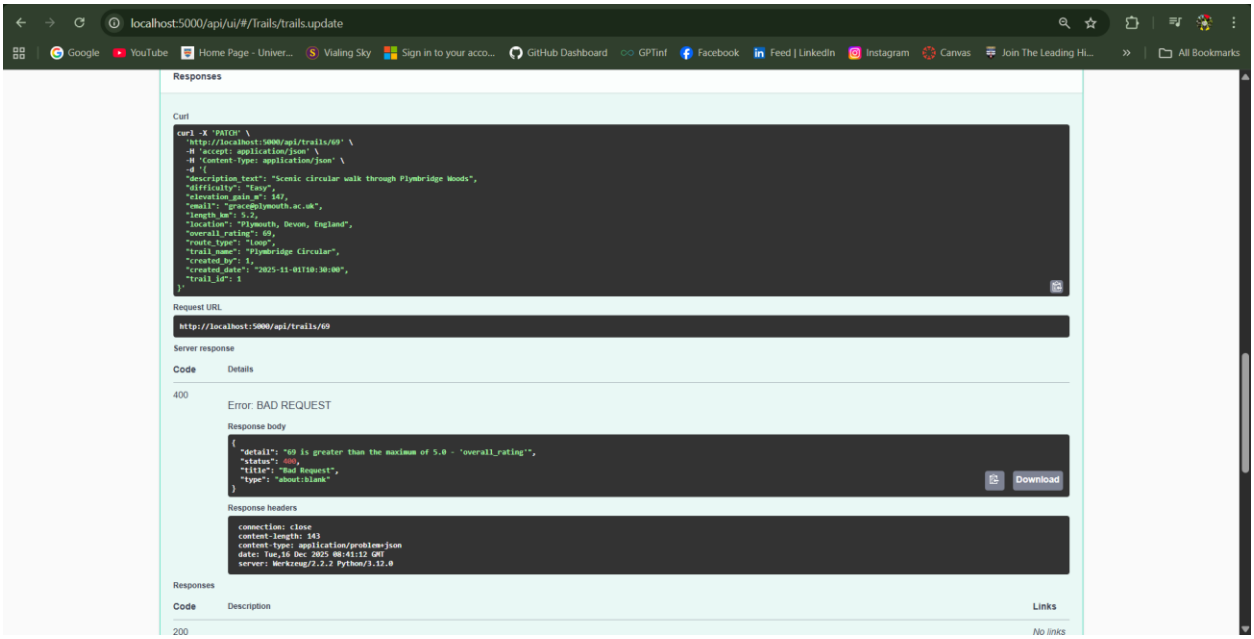

Figure 48 : TC-A14(b)

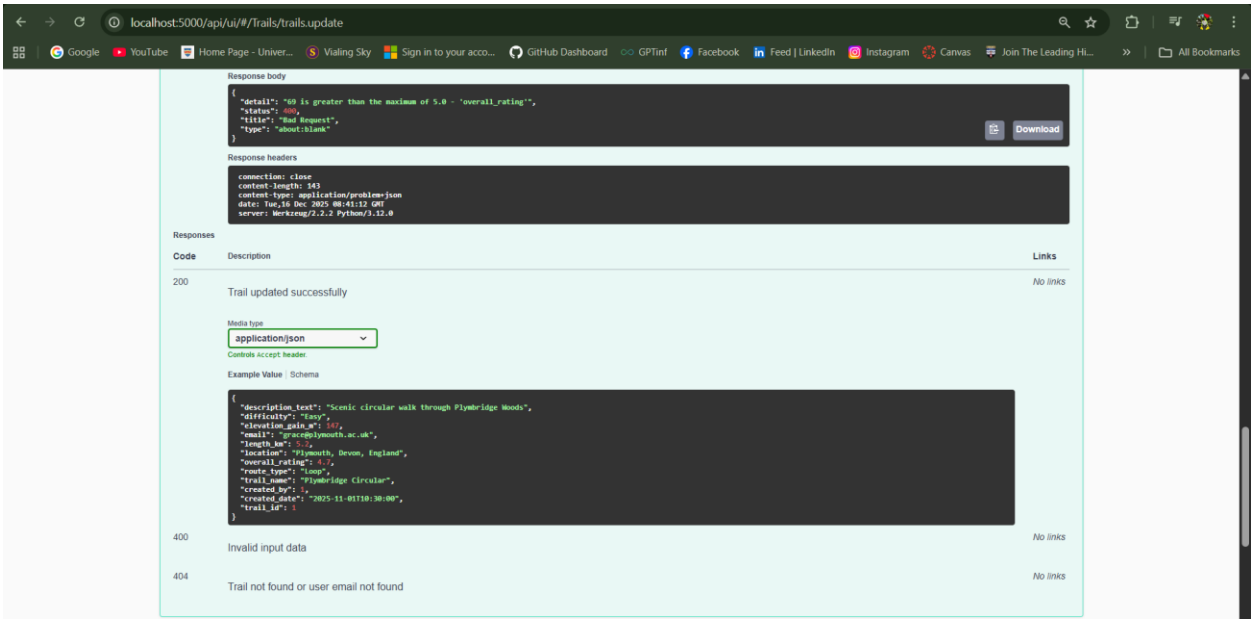

Figure 49 : TC-A14(c)

- HTTP **400 Bad Request**
- Trail remains unchanged

TC-A15: Update Non-Existent Trail

Endpoint: `PATCH /api/trails/{trail_id}`

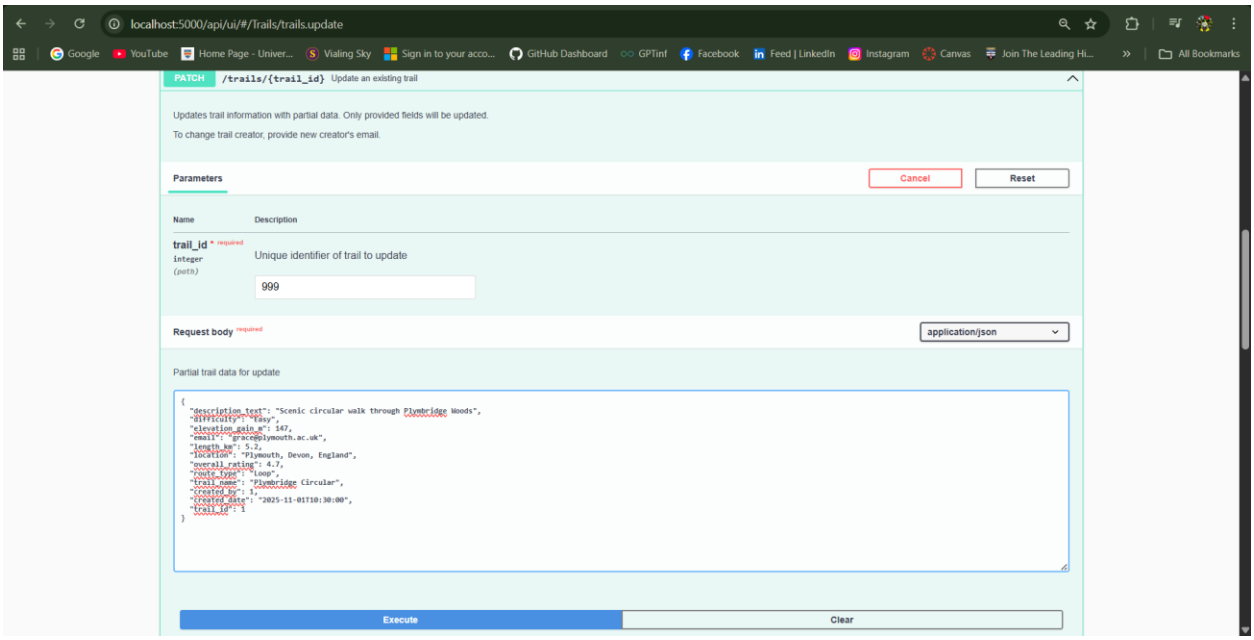

Figure 50 : TC-A15(a)

Purpose

To validate error handling for missing trail records.

After

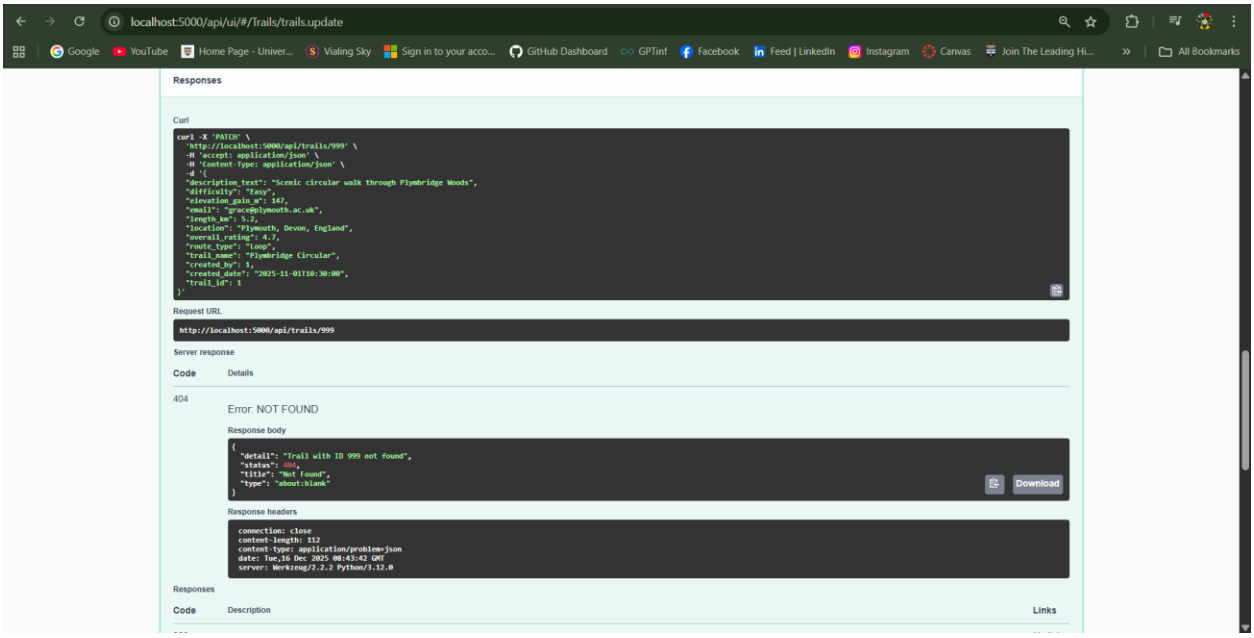

Figure 51 : TC-A15(b)

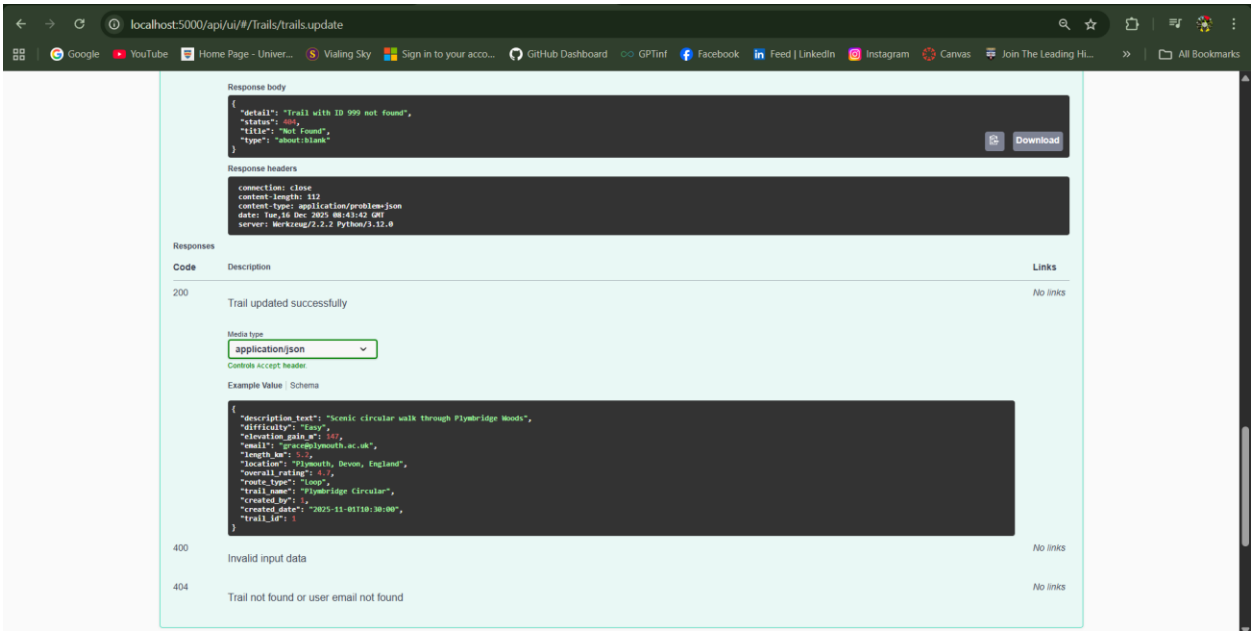

Figure 52 : TC-A15(c)

- HTTP **404 Not Found**

TC-A16: Create Trail with Invalid Numeric Boundary

Endpoint: `POST /api/trails`

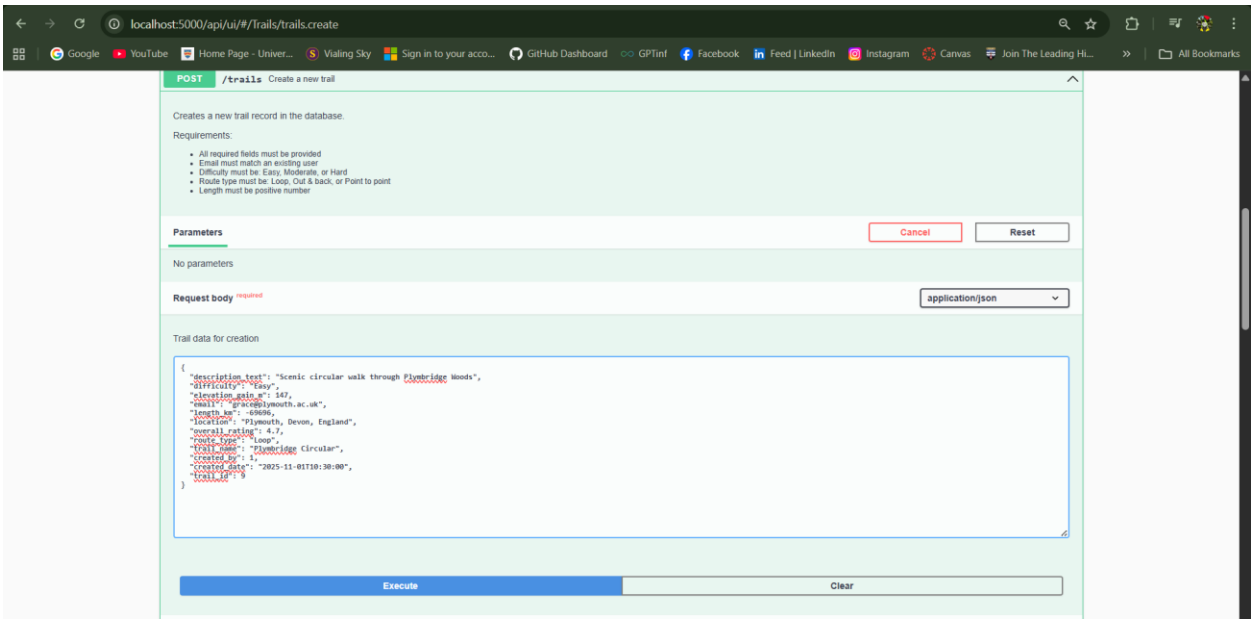

Figure 53 : TC-A16(a)

Purpose

To confirm enforcement of numeric constraints such as minimum length_km.

After

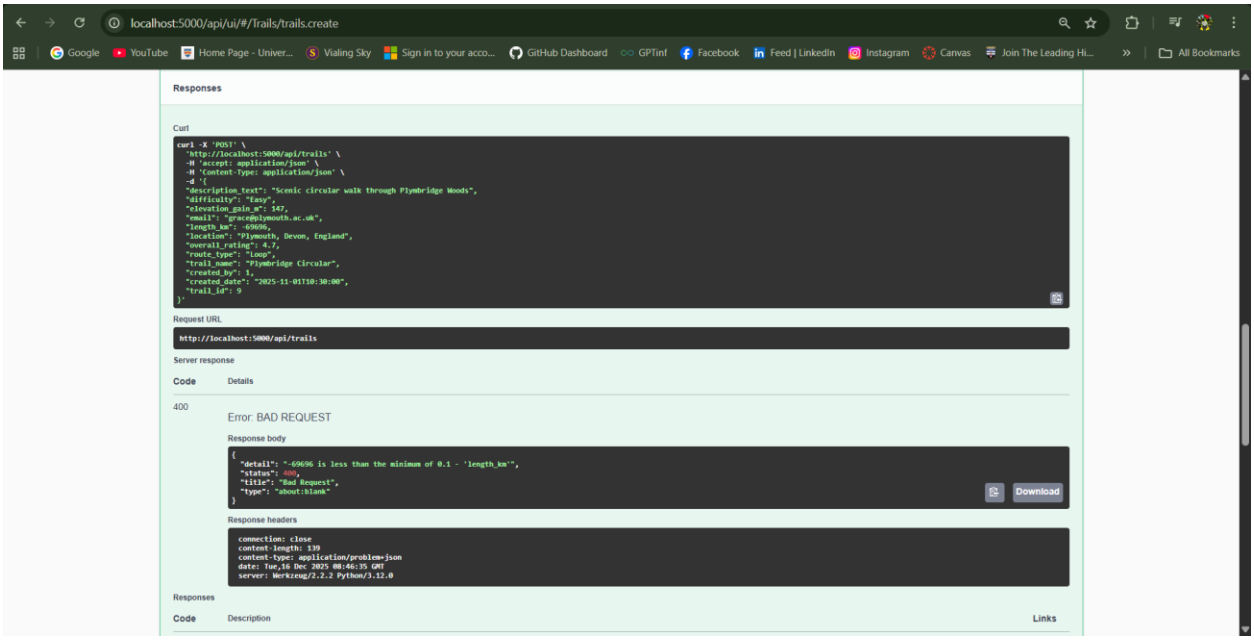

Figure 54 : TC-A16(b)

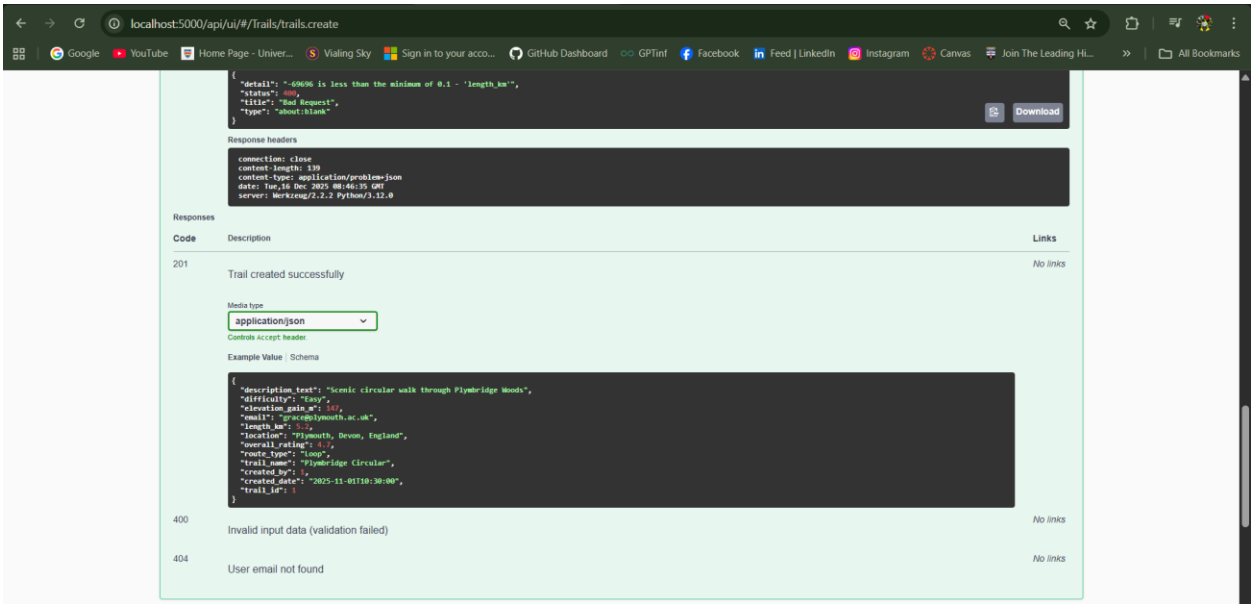

Figure 55 : TC-A16(c)

- HTTP **400 Bad Request**
- Numeric validation error returned

TC-A17: Retrieve Features for Trail with No Features

Endpoint: `GET /api/trails/{trail_id}/features`

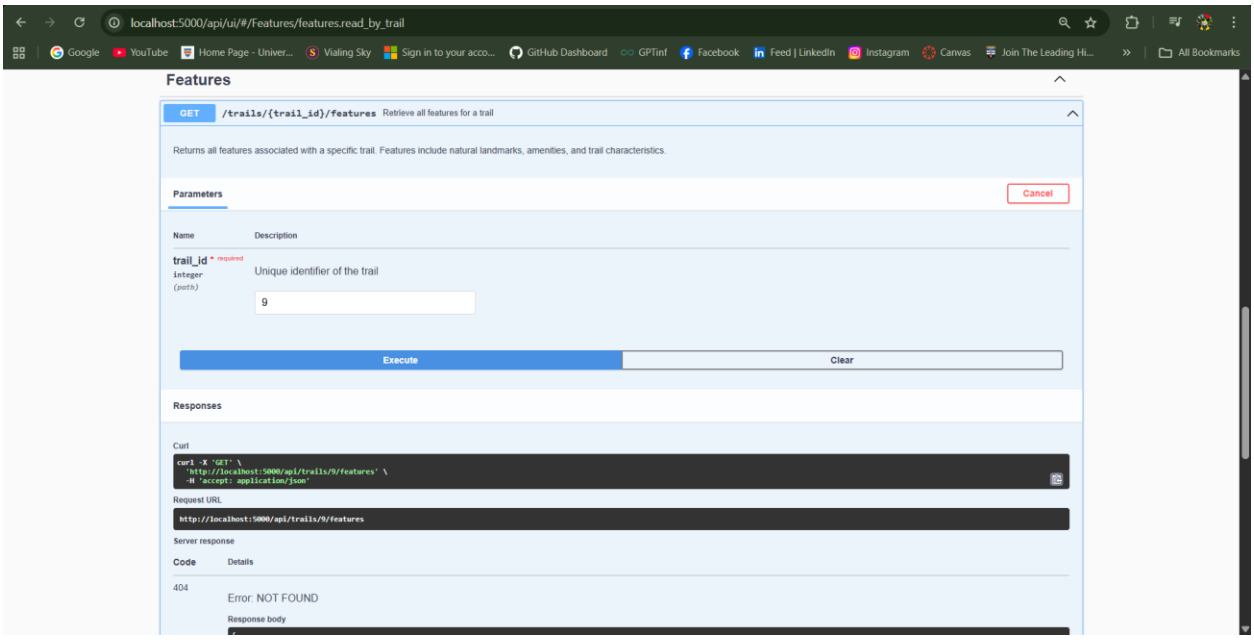

Figure 56 : TC-A17(a)

Purpose

To ensure consistent handling of valid trails with no feature associations.

After

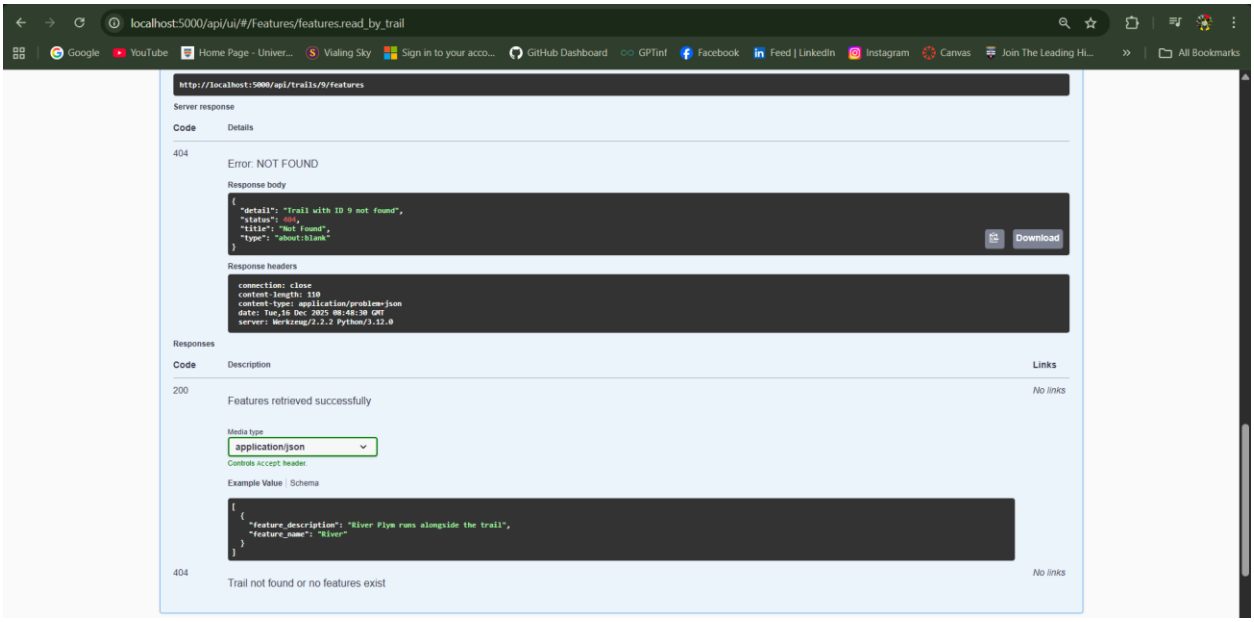

Figure 57 : TC-A17(b)

- HTTP **404 Not Found**
- Informative message returned

TC-A18: Delete Non-Existent Trail

Endpoint: `DELETE /api/trails/{trail_id}`

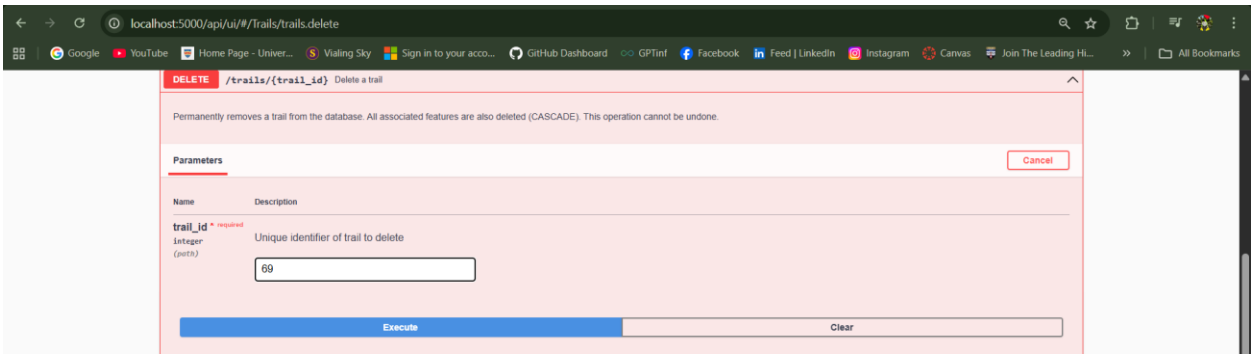

Figure 58 : TC-A18(a)

Purpose

To validate correct error handling when deleting an invalid resource.

After

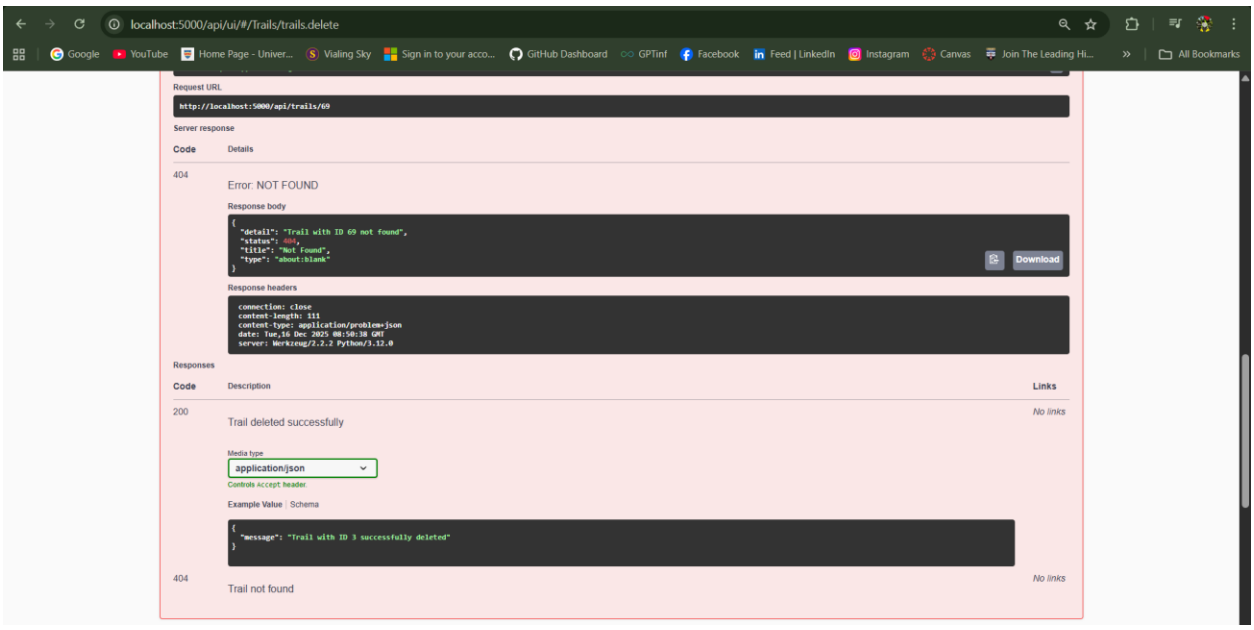

Figure 59 : TC-A18(b)

- HTTP **404 Not Found**

6.0 Evaluation and Reflection

The test suite validated functional correctness across all implemented endpoints. Every CRUD operation performed as expected with appropriate status codes and response formats.

One interesting discovery during testing was the difference in performance between direct table queries and view queries. The vw_TrailsWithCreator view which performs a JOIN between TRAIL and USER tables that executed slightly slower than the equivalent ORM query using SQLAlchemy's relationship loading which suggests the database query optimizer isn't necessarily smarter than SQLAlchemy's query generation at least for simple joins.

Moreover, the combination of Connexion's schema validation and explicit checks in controller code caught all invalid inputs before they reached the database.

Somehow, most significant weakness is lack of authentication and authorization. For example, the API is functionally complete but essentially unusable in any realistic deployment without access control. Thus, implementing pagination for list endpoints would be the most impactful improvement. Adding filtering and sorting capabilities would increase API utility. Audit logging could expand beyond trail creation to capture all modifications with complete historical records.

7.0 Conclusion

Coursework 2 shows how a database design from CW1 can be carried through into functioning RESTful API while also exposing the practical frictions that only appear during implementation. The TrailService behaves as intended like endpoints are consistent and the API responds predictably to both valid and invalid requests. Using a microservice-style boundary helped keep the problem focused, although it also made certain gaps, especially around privacy etc. In that sense, the design works but it also highlights what has been left out.

Looking back, the most valuable outcome may not be the API itself but the clearer understanding of trade-offs. Convenience sometimes won over purity such as returning nested creator details or relying on ORM-generated queries instead of stored procedures. These choices are defensible, though not universally “correct,” and in a larger system they would likely be revisited. Recognising where the system is solid, where it is fragile and why those boundaries exist feels like a step toward more mature design.

8.0 References

Alspach, K. (2019) *Rate Limiting: A Useful Tool for Managing API Traffic*. Boston, MA: Boston Consulting Group.

Baeldung (2024) *Database Design in a Microservices Architecture*. Available at: https://www.baeldung.com/cs/microservices-db-design (Accessed: 20 November 2025).

EdPrice-MSFT (2023) *Web API design best practices - Azure Architecture Center*. Available at: https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design (Accessed: 21 November 2025).

Kleppmann, M. (2017) *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media.

Newman, S. (2021) *Building Microservices: Designing Fine-Grained Systems*. 2nd edn. Sebastopol, CA: O'Reilly Media.

OWASP (2017) *OWASP Top 10 - 2017: The Ten Most Critical Web Application Security Risks*. OWASP Foundation.

Prometheus (2023) *Prometheus Monitoring System*. Available at: https://prometheus.io/ (Accessed: 14 December 2025).

Relevant Software (2025) *All You Need to Know About Microservices Database Management*. Available at: https://relevant.software/blog/microservices-database-management/ (Accessed: 22 November 2025).

Richardson, C., Smith, F. and Floyd, J. (2018) *Microservices Patterns: With Examples in Java*. Shelter Island, NY: Manning Publications.

9.0 Appendix

Appendix A : Initial ERD from CW1

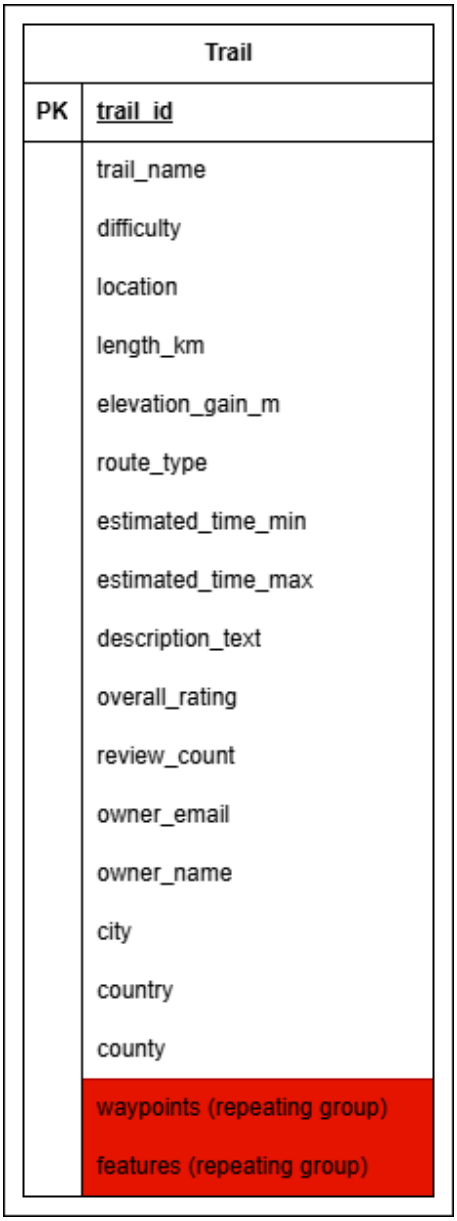

Figure 60: Initial Entity Relationship Diagram

Assumptions about Initial ERD:

Trail is the central enity containing all trail information. However, the waypoints and features are stored as repeating groups. Then, the owner information is embedded within the trail entity. Last, location data such as city, county, country is denormalized.

Appendix B : Partial ERD from CW1

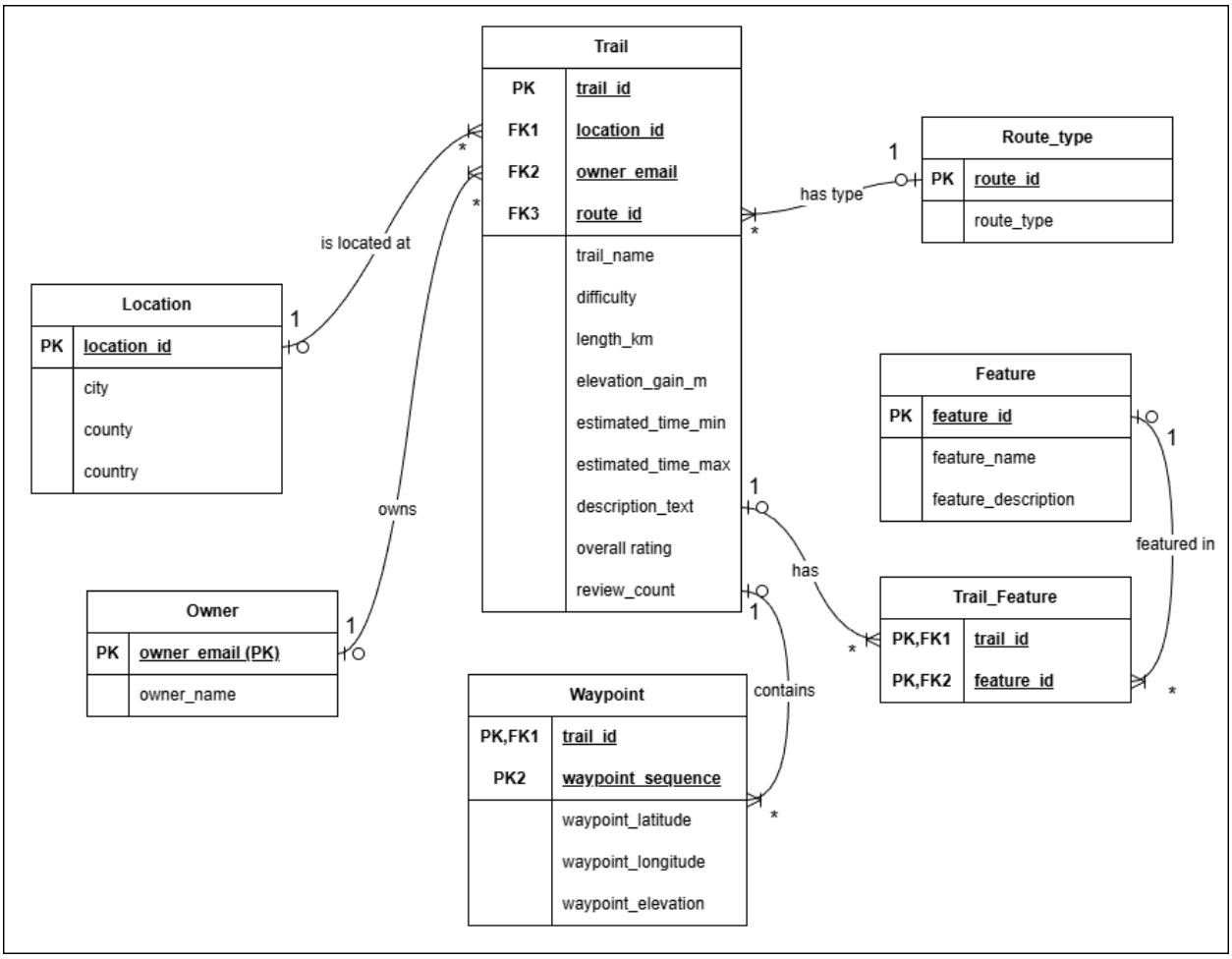

Figure 61 : Partial Entity Relationship Diagram

Appendix C : Field Definition Grids from CW1

Table 1: USER

Field Name	Data Type	Size	Constraints	Description
user_id	INT	-	PK, IDENTITY(1,1)	Unique user identifier
Email	NVARCHAR	255	NOT NULL, UNIQUE	User email (matches auth API)
Username	NVARCHAR	100	NOT NULL	Display name
full_name	NVARCHAR	255	NULL	User's full name
Role	NVARCHAR	50	NOT NULL, DEFAULT 'user'	User role (admin/user)
registration_date	DATETIME	-	NOT NULL, DEFAULT GETDATE()	Account creation timestamp

Table 2: TRAIL

Field Name	Data Type	Size	Constraints	Description
trail_id	INT	-	PK, IDENTITY(1,1)	Unique trail identifier
trail_name	NVARCHAR	255	NOT NULL	Trail name
difficulty	NVARCHAR	50	NOT NULL, CHECK	Easy/Moderate/Hard
location	NVARCHAR	255	NOT NULL	Trail location description
length_km	DECIMAL	10,2	NOT NULL, CHECK > 0	Trail length in kilometers
elevation_gain_m	INT	-	NULL, CHECK >= 0	Elevation gain in meters
route_type	NVARCHAR	50	NOT NULL	Loop/Out & back/Point to point
description_text	NVARCHAR	MAX	NULL	Trail description
overall_rating	DECIMAL	3,2	NULL, CHECK 1-5	Average rating

Field Name	Data Type	Size	Constraints	Description
created_by	INT	-	NOT NULL, FK → USER	Trail creator user_id
created_date	DATETIME	-	NOT NULL, DEFAULT GETDATE()	Creation timestamp

Table 3: TRAIL_FEATURE

Field Name	Data Type	Size	Constraints	Description
trail_id	INT	-	PK, FK → TRAIL	Reference to trail
feature_name	NVARCHAR	100	PK	Feature name
feature_description	NVARCHAR	500	NULL	Feature description

Table 4: TRAIL_LOG (for Trigger)

Field Name	Data Type	Size	Constraints	Description
log_id	INT	-	PK, IDENTITY(1,1)	Unique log identifier
trail_id	INT	-	NOT NULL	Trail that was added
trail_name	NVARCHAR	255	NOT NULL	Name of trail added
added_by	INT	-	NOT NULL	User who added trail
added_by_email	NVARCHAR	255	NOT NULL	Email of user
timestamp	DATETIME	-	NOT NULL, DEFAULT GETDATE()	When trail was added